

Analisi e Progettazione del Software

Verso l'analisi a oggetti e la modellazione di dominio

Oliviero Riganelli

Iterazioni del corso e iterazioni di un progetto reale

Corso

- Iterazione 1 guidata dagli obiettivi di apprendimento
- Iterazione 1 non è centrata sull'architettura né guidata dal rischio



Progetto reale

- Iterazione 1 guidata dagli obiettivi di un progetto
- Iterazione 1 affronta prima gli aspetti difficili e rischiosi

Nel contesto del corso che ha l'obiettivo di aiutare ad apprendere i concetti base dell'OOA/D e di UML, si è deciso di iniziare con degli argomenti più semplici

NextGen POS

Requisiti per l'iterazione 1

- *Implementare uno scenario di base del caso d'uso **Elabora Vendita**: inserire gli articoli e ricevere un pagamento in contanti*
- *Implementare un caso d'uso di **Avviamento**, necessario per gestire le esigenze di inizializzazione per questa iterazione*

Non c'è niente di strano o di complesso da gestire:

- *Uno scenario di successo*
- *Non c'è collaborazione con i servizi esterni, come una base di dati dei prodotti oppure il sistema esterno di contabilità.*
- *Non vengono applicate regole complesse per la determinazione dei prezzi*

Monopoly

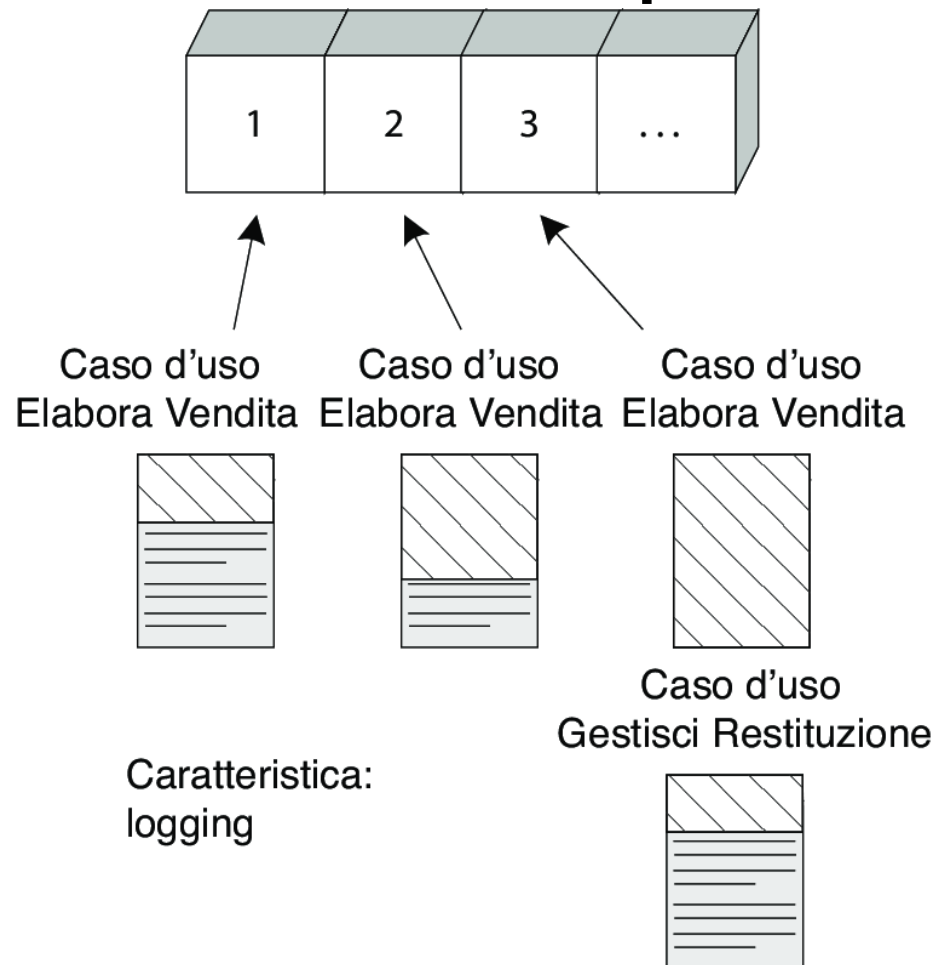
Requisiti per l'iterazione 1:

- Implementare uno scenario di base del caso d'uso **Gioca una Partita a Monopoly**: i giocatori si spostano fra le caselle del tabellone
- Implementare un caso d'uso di **Avviamento** necessario per gestire le esigenze di inizializzazione per questa iterazione
- Possono giocare da 2 a 8 giocatori
- Una partita consiste in una serie di 20 giri.
 - Durante un giro, ogni giocatore gioca turno.
 - In un turno, un giocatore lancia due dati (a sei facce) e fa avanzare il proprio segnalino lungo il tabellone
 - Vengono visualizzati il nome del giocatore e il nome della casella su cui il giocatore si è fermato

Non c'è niente di strano o di complesso da gestire:

- Non ci sono soldi, non ci sono né vincitori né perdenti, nessuna proprietà da acquistare o affitti da pagare e nessuna casella speciale di alcun genere

Sviluppo incrementare per uno stesso caso d'uso in più iterazioni



Un caso d'uso o una caratteristica sono spesso troppo complessi per poter essere completati in una sola breve iterazione.

Pertanto le varie parti o scenari possono essere distribuiti su diverse iterazioni.

Processo: Ideazione ed Elaborazione

Cosa è successo durante ideazione?

- Un breve workshop dei requisiti
- Assegnati i nomi per la maggior parte degli attori, obiettivi e casi d'uso
- molti casi d'uso in formato breve e alcuni casi d'uso in formato dettagliato
- identificati la maggior parte dei requisiti di qualità più influenti e rischiosi
- bozza della Visione e delle Specifiche supplementari
- lista dei rischi
- Prototipi tecnici proof-of-concept
- Prototipi orientati all'UI per chiarire la visione dei requisiti funzionali
- Proposta dell'architettura di alto livello e dei componenti candidati
- Piano per la prima iterazione dell'**Elaborazione**

Verso l'elaborazione

L'elaborazione è la serie iniziale di iterazioni durante le quali, in un progetto normale:

- *Viene programmato e verificato il nucleo, rischioso, dell'architettura software*
- *Viene scoperta e stabilizzata la maggior parte dei requisiti*
- *I rischi maggiori sono attenuati o rientrano*

L'elaborazione è spesso costituita da due o più iterazioni che durano ognuna dalle 2 alle 6 settimane (ogni iterazione è timeboxed)

L'elaborazione in una sola frase:

Costruire il nucleo dell'architettura, risolvere gli elementi ad alto rischio, definire la maggior parte dei requisiti e stimare il piano di lavoro e le risorse complessive

Alcune idee e best practice fondamentali per l'elaborazione

- Eseguire iterazioni guidate dal rischio, brevi e timeboxed
- Iniziare presto la programmazione
- Progettare, implementare e testare, in modo attivo, le parti principali e più rischiose dell'architettura
- Effettuare test presto, spesso e in modo realistico
- Adattare in base al feedback proveniente da test, utenti e sviluppatori
- Scrivere la maggior parte dei casi d'uso e degli altri requisiti nel dettaglio
 - *un workshop dei requisiti per iterazione*

Elaborati iniziati durante l'elaborazioni (esclusi quelli iniziati durante l'ideazione)

Elaborato	Commento
Modello di Dominio	È una visualizzazione dei concetti del dominio, simile a un modello statico delle informazioni delle entità del dominio.
Modello di Progetto	È l'insieme dei diagrammi che descrivono la progettazione logica. Comprende diagrammi delle classi software, diagrammi di interazione degli oggetti, diagrammi dei package e così via.
Documento dell'Architettura Software	Un aiuto per l'apprendimento che riassume gli aspetti principali dell'architettura e la loro risoluzione nel progetto. È un riepilogo delle idee di progettazione più significative all'interno del sistema e delle loro motivazioni.
Modello dei Dati	Comprende gli schemi della base di dati e le strategie di mapping tra la rappresentazione a oggetti e la base di dati.
Storyboard dei casi d'uso, Prototipi UI	Una descrizione dell'interfaccia utente, della navigazione, dei modelli di usabilità e così via.

Pianificare l'interazione successiva

I requisiti e le iterazioni vanno organizzate in base al rischio, copertura e criticità:

- **Rischio:** *comprende tanto la complessità tecnica quanto altri fattori, come l'incertezza dello sforzo o l'usabilità*
- **Copertura:** *indica che le iterazioni iniziali prendono in considerazione tutte le parti principali del sistema*
- **Criticità (valore):** *si riferisce alle funzioni che il cliente considera di elevato valore di business*

Ordinati con tecniche di votazione

- *Le prime iterazioni implementano gli scenari con il voto maggiore*

Pianificazione adattiva e iterativa

Pianificare l'iterazione successiva

Voto	Requisito (Caso d'uso o Caratteristica)	Commento
Alto	Elabora Vendita Logging	Ottiene voti alti per tutti i criteri. Pervasivo. Difficile da aggiungere in un secondo momento.
	...	...
Medio	Gestire utenti	Influisce sul sottodominio di sicurezza.
	...	...
Basso	...	...

La classifica viene stilata prima dell'iterazione 1, ma poi nuovamente prima dell'iterazione 2 e così via, man mano che i nuovi requisiti e nuove intuizioni influenzano l'ordine.

Non hai capito l'elaborazione se ...

- Ha una durata superiore ad “alcuni” mesi per la maggior parte dei progetti
- Ha una sola iterazione (con rare eccezioni per problemi di chiara comprensione)
- la maggior parte dei requisiti sono stati definiti prima dell'elaborazione
- le fondamenta dell'architettura e gli elementi rischiosi non vengono affrontati
- non produce una architettura eseguibile; il codice non è di qualità di produzione
- è considerata principalmente una fase di analisi dei requisiti, che precede una fase di implementazione, la costruzione
- prova a fare una progettazione completa prima dell'implementazione
- feedback e adattamento minimi; gli utenti non sono coinvolti in modo continuo
- non c'è test precoce e realistico
- è considerata una fase in cui vengono prodotti prototipi per dimostrare idee, e non la baseline dell'architettura eseguibile
- non ci sono vari brevi workshop dei requisiti, che adattano e raffinano i requisiti basandosi sul feedback delle iterazioni precedenti e corrente

Verso l'analisi a oggetti

Analizzare e modellare

- Il **Dominio informativo**, ovvero le tipologie di informazioni che il sistema deve rappresentare e gestire
- Le interazioni fra gli attori e il sistema, ovvero le **funzioni** che il sistema è chiamato a svolgere durante il suo uso
- Il **comportamento** del sistema ovvero i cambiamenti nelle informazioni associati a ciascuna funzione

Nell'analisi orientata agli oggetti, questi tre aspetti vengono modellati come segue

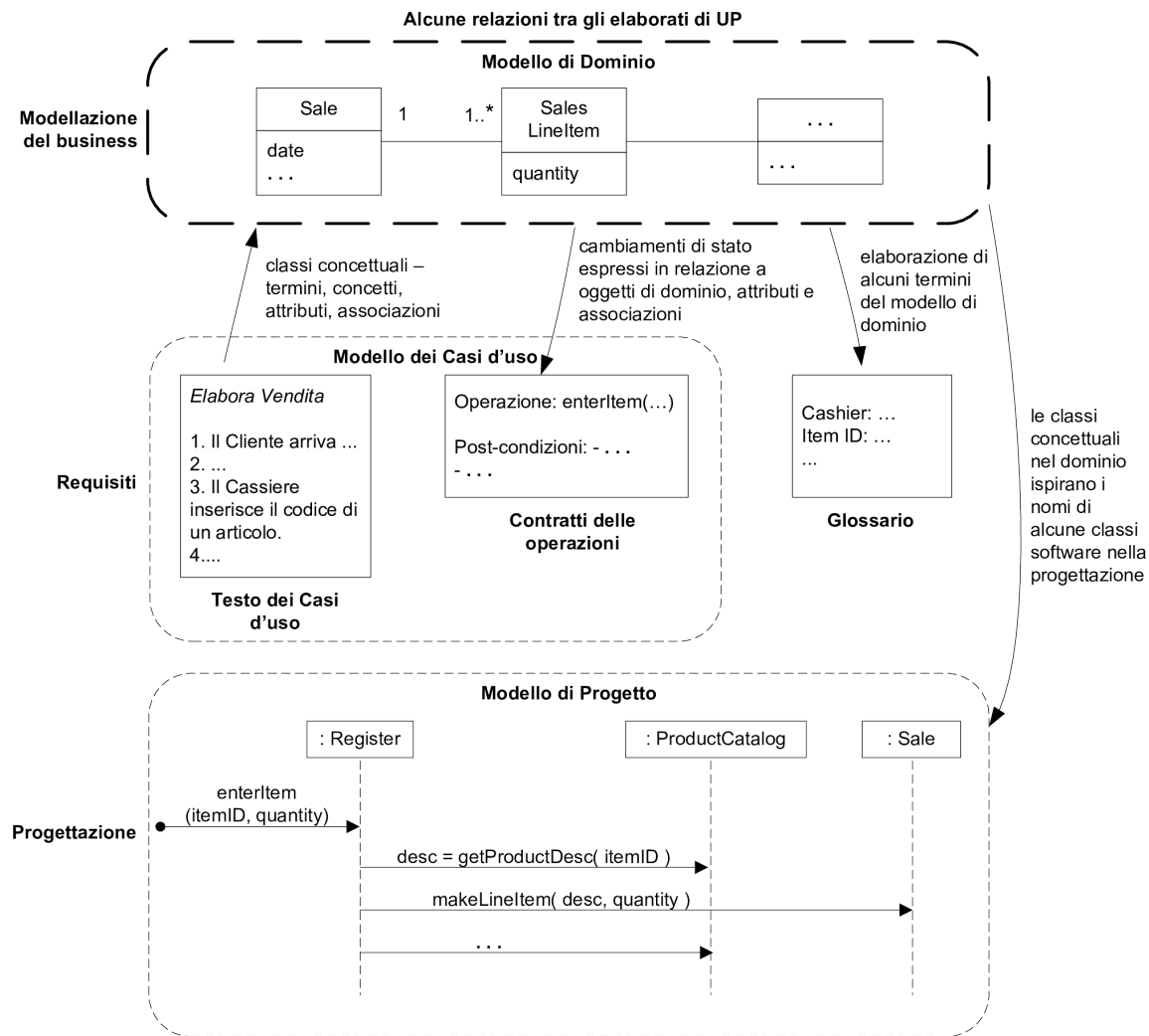
- Il dominio informativo è rappresentato mediante un modello oggetti (**modello di dominio**)
- Le funzioni del sistema sono rappresentate in termini delle operazioni che il sistema è chiamato a svolgere (**operazioni di sistema**), insieme a una descrizione dell'ordine relativo in cui si possono richiedere queste operazioni (**diagrammi di sequenza di sistema**)
- Il comportamento il sistema è descritto come l'effetto prodotto dall'esecuzione di ciascuna operazione sistema, in termini cambiamenti le informazioni del modello di dominio (**contratti** delle operazioni di sistema)

Modellazione di Dominio

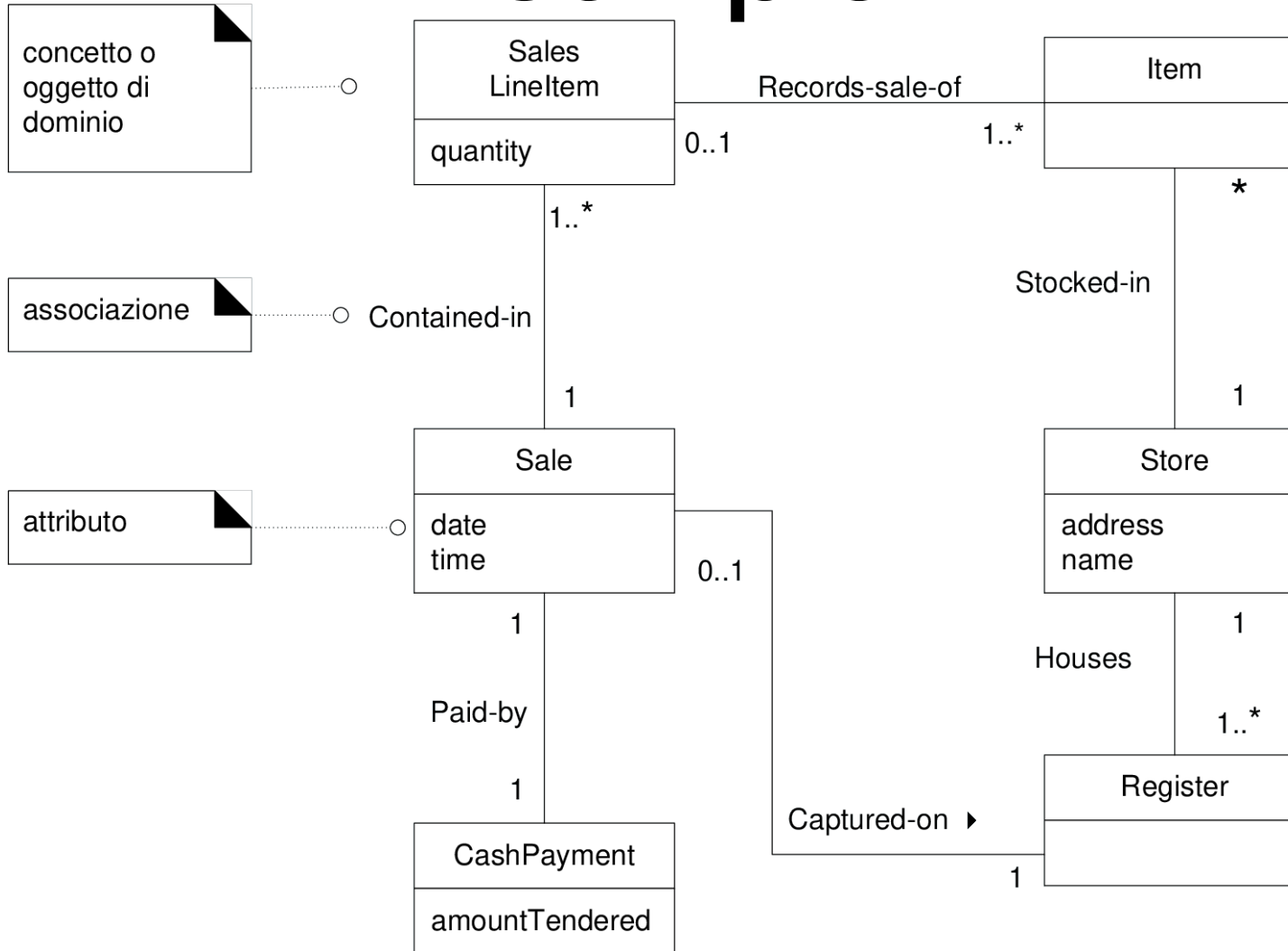
Un modello di dominio è il modello più importante dell'OOA

- Descrive le classi concettuali e le relazioni tra esse
- Fonte di ispirazione per classi di progetto e di implementazione
- Sviluppato in modo iterativo ed incrementale
- Limitato dai requisiti dell'iterazione corrente

Influenza tra alcuni elaborati



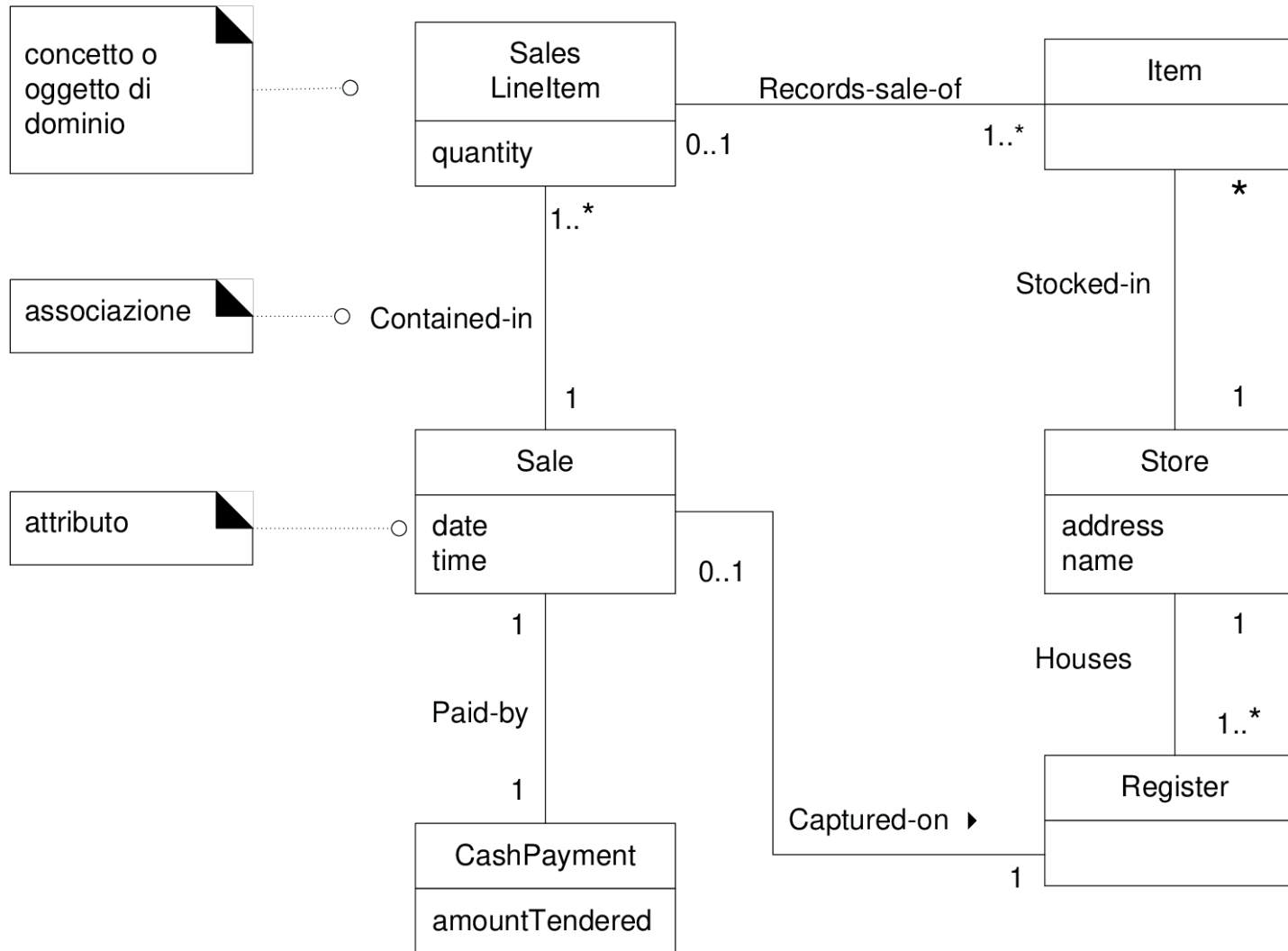
Esempio



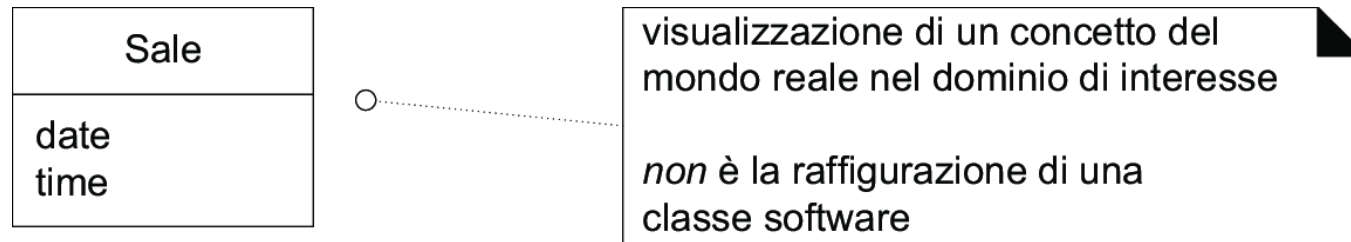
Che cos'è un modello di dominio

- Un modello di dominio è una rappresentazione visuale di classi concettuali o di oggetti del mondo reale, nonché delle relazioni tra di essi, in un dominio di interesse
- Applicando UML, un modello di dominio può essere realizzato come uno o più diagrammi delle classi in cui non sono definite operazioni e mostrano:
 - *Classi concettuali o oggetti di dominio*
 - *associazioni tra classi concettuali*
 - *attributi di classi concettuali*

Il modello di dominio è un “dizionario visuale”



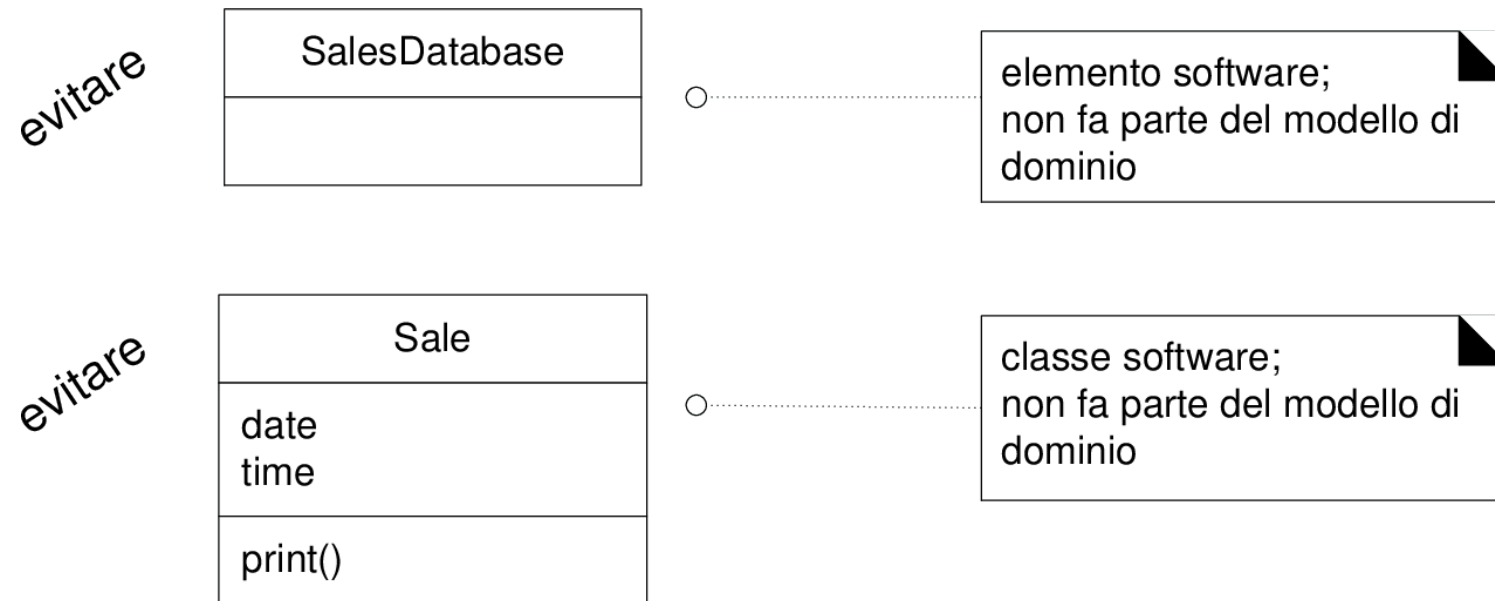
Il modello di dominio non è una raffigurazione di oggetti software



Un modello di dominio è un dizionario visuale delle **astrazioni** significative, della terminologia del dominio e del contenuto informativo del dominio di interesse

- *mostra i vocaboli e la struttura del linguaggio proprio del dominio di interesse*
- *le classi concettuali sono astrazioni di oggetti reali*

Il modello di dominio non è una raffigurazione di oggetti software



Un modello di dominio non mostra *elementi software, metodi o responsabilità*

Due significati tradizionali di modello di dominio

- in UP
 - *Descrive da un punto di vista concettuale gli oggetti del mondo reale*
- in Smalltalk
 - *Indica lo strato di oggetti software del domino sotto lo strato UI*
- Quale delle due definizioni è corretta?

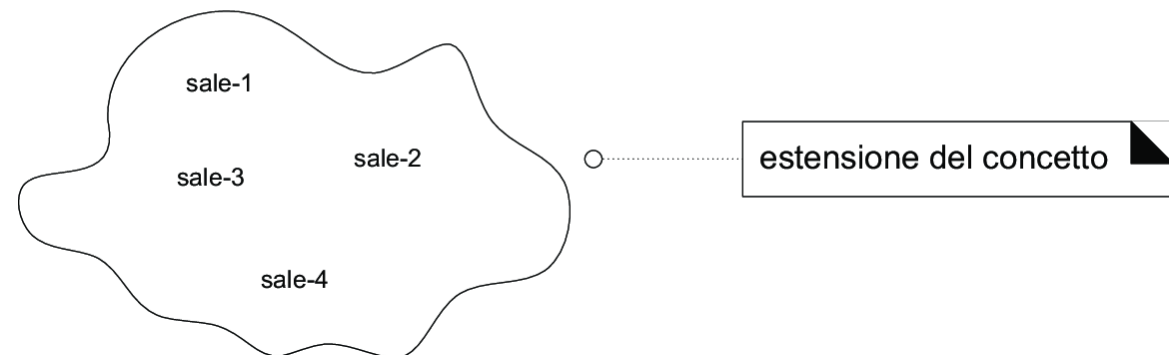
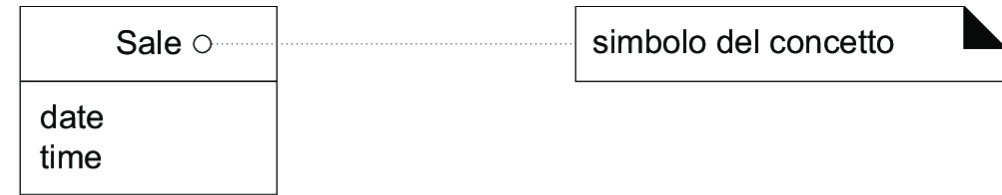
Modello di dominio e modello di dati

- Un modello dei dati mostra i dati che devono essere memorizzati in modo persistente
- Un modello di dominio descrive informazioni che devono essere gestite nel sistema in discussione e può contenere
 - *classi concettuali senza attributi*
 - *classi concettuali che hanno un ruolo puramente comportamentale e non un ruolo informativo*

Classi concettuali

Una classe concettuale è un'*idea*, una *cosa* o un *oggetto* che può essere considerata in termini di

- **simbolo**: una parola o un'immagine usata per rappresentare la classe concettuale
- **intenzione**: la definizione (linguaggio naturale) della classe concettuale
- **estensione**: l'insieme degli oggetti **descritti** dalla classe concettuale



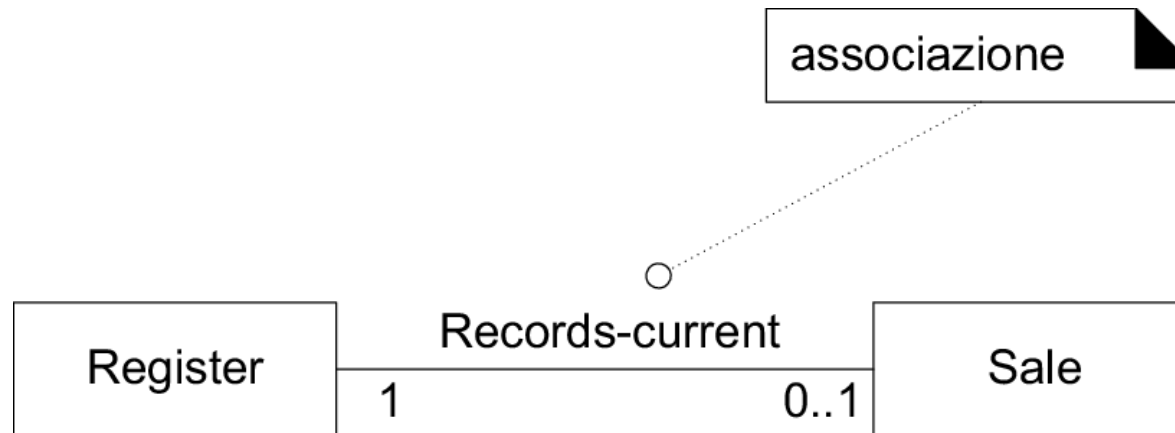
Classe

- In UML, una classe è il descrittore per un insieme di oggetti che possiedono le stesse caratteristiche (attributi, operazioni, metodi, relazioni e comportamento)
- Una classe concettuale rappresenta un concetto del mondo reale e un insieme di oggetti che possiedono caratteristiche strutturali (attributi e relazioni) simili



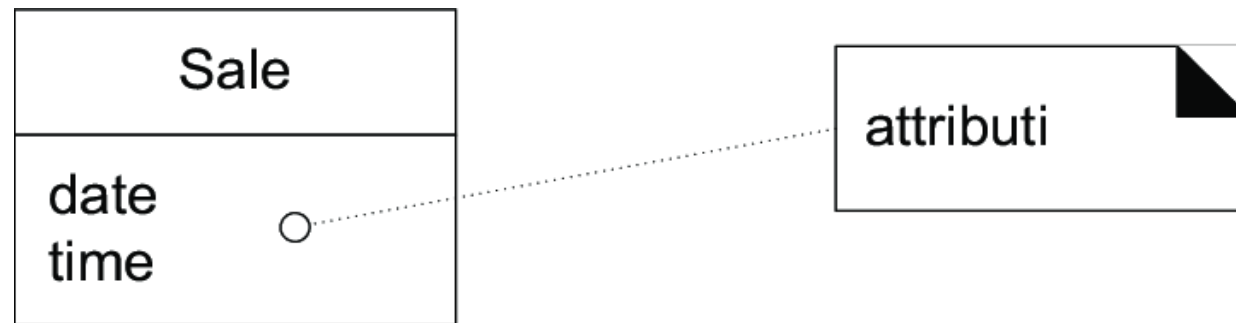
Associazioni

- In UML, un'associazione è la relazione tra due o più classificatori che comporta connessioni tra le rispettive istanze
- Un'associazione è una relazione tra classi (più precisamente, tra le istanze di queste classi) che indica una connessione significativa e interessante



Attributi

- In UML, un attributo è la descrizione di una proprietà in una classe
- Un attributo è un valore logico (ovvero un dato, una proprietà elementare) degli oggetti di una classe



Perchè creare un modello di dominio

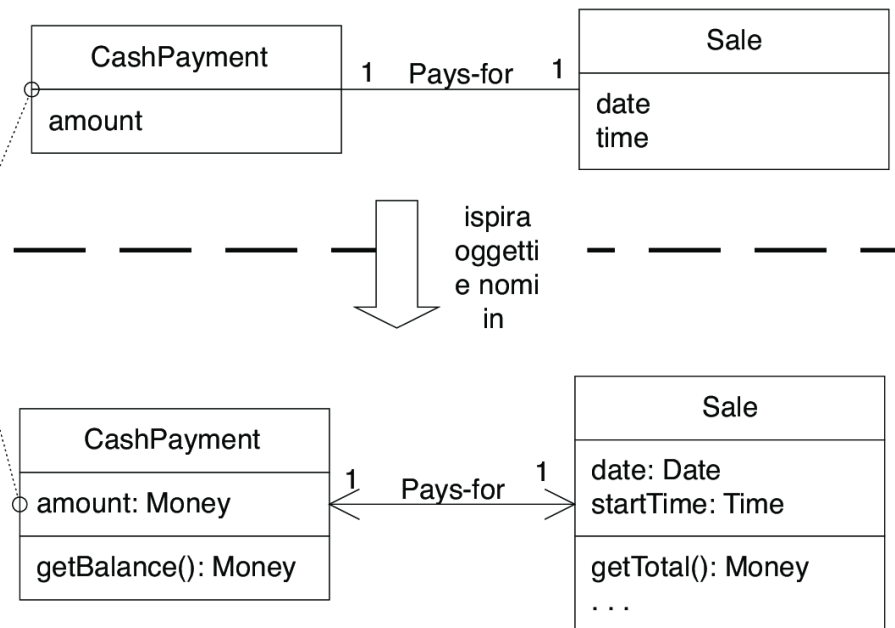
- per comprendere il (dominio del) sistema da realizzare e il suo vocabolario (analisi)
- fonte di ispirazione per lo strato del dominio

CashPayment nel Modello di Dominio è un concetto, ma CashPayment nel Modello di Progetto è una classe software. Non sono la stessa cosa, ma il primo ha *ispirato* il nome e la definizione del secondo.

Questa scelta riduce il salto rappresentazionale.

Questa è una delle grandi idee della tecnologia a oggetti.

Modello di Dominio di UP
Vista delle parti interessate dei concetti significativi del dominio.



Modello di Progetto di UP

Lo sviluppatore orientato agli oggetti ha tratto ispirazione dal dominio del mondo reale per la creazione di classi software.

Pertanto, il salto rappresentazionale tra il modo in cui le parti interessate concepiscono il dominio e la sua rappresentazione nel software è stato tenuto basso.

Come creare un modello di dominio

Rimanendo vincolati ai requisiti scelti per la progettazione nell'interazione corrente, si devono affrontare i seguenti passi

1. Trovare le classi concettuali
2. Disegnarle come classi in un diagramma delle classi UML
3. Aggiungere associazioni e attributi

Identificare le classi concettuali

Tre strategie per trovare le classi concettuali

1. Riusare o modificare dei modelli esistenti
2. Utilizzare un elenco di categorie comuni
3. Identificare nomi e locuzioni nominali

Utilizzare un elenco di categorie comuni

Categoria di classe concettuale	Esempi
transazioni commerciali <i>Linea guida:</i> sono aspetti critici (riguardano denaro), dunque si inizi con le transazioni.	<i>Sale, Payment (o CashPayment)</i> <i>Reservation</i>
elementi/righe di transazioni <i>Linea guida:</i> le transazioni spesso sono composte da righe per gli articoli correlati, quindi queste vanno considerate subito dopo le transazioni.	<i>SalesLineItem</i>
prodotto o servizio correlato a una transazione o a una riga di transazione per articolo <i>Linea guida:</i> le transazioni sono per qualcosa (un prodotto o un servizio). Vanno considerate subito dopo.	<i>Item</i> <i>Flight, Seat, Meal</i>
dove viene registrata la transazione? <i>Linea guida:</i> importante.	<i>Register, Ledger (o SalesLedger), FlightManifest</i>

Utilizzare un elenco di categorie comuni

ruoli di persone o organizzazioni correlati alle transazioni; attori nei casi d'uso <i>Linea guida:</i> normalmente dobbiamo sapere quali sono le parti coinvolte in una transazione.	<i>Cashier, Customer, Store MonopolyPlayer Passenger, Airline</i>
luogo della transazione; luogo del servizio	<i>Store Airport, Plane, Seat</i>
eventi significativi, spesso con un'ora o un luogo che è necessario ricordare	<i>Sale, Payment (o CashPayment) MonopolyGame Flight</i>
oggetti fisici <i>Linea guida:</i> questo è particolarmente importante quando si crea software per il controllo di dispositivi, oppure simulazioni.	<i>Item, Register Board, Piece, Die Airplane</i>
descrizioni di oggetti <i>Linea guida:</i> vedere il Paragrafo 12.13 per una discussione.	<i>ProductDescription FlightDescription</i>

Utilizzare un elenco di categorie comuni

cataloghi <i>Linea guida:</i> le descrizioni sono spesso contenute in un catalogo.	<i>ProductCatalog FlightCatalog</i>
contenitori di oggetti (fisici o informazioni)	<i>Store, Bin Board Airplane</i>
oggetti in un contenitore	<i>Item Square (in un Board) Passenger</i>
altri sistemi che collaborano	<i>CreditAuthorizationSystem AirTrafficControl</i>
registrazioni di questioni finanziarie, di lavoro, contrattuali e legali	<i>Receipt, Ledger MaintenanceLog</i>
strumenti finanziari	<i>Cash, Check, LineOfCredit TicketCredit</i>
piani, manuali, documenti cui si fa regolarmente riferimento per eseguire il lavoro	<i>DailyPriceChangeList RepairSchedule</i>

Identificare nomi e locuzioni nominali

Scenario principale di successo (o Flusso di base):

1. Il **Cliente** arriva alla **cassa POS** con gli **articoli** e/o i **servizi** da acquistare.
2. Il **Cassiere** inizia una nuova **vendita**.
3. Il **Cassiere** inserisce il **codice identificativo di un articolo**.
4. Il Sistema registra la **riga di vendita per l'articolo** e mostra la **descrizione dell'articolo**, il suo **prezzo**, il **totale parziale**. Il prezzo è calcolato in base a un insieme di regole di prezzo.

Il Cassiere ripete i passi 2-3 fino a che non indica che ha terminato.

5. Il Sistema mostra il totale con le **imposte** calcolate.
6. Il Cassiere riferisce il totale al Cliente e richiede il **pagamento**.
7. Il Cliente paga e il Sistema gestisce il pagamento.
8. Il Sistema registra la vendita completata e invia informazioni sulla **vendita** e sul pagamento ai sistemi esterni di **Contabilità** (per la contabilità e le **commissioni**) e di **Inventario** (per l'aggiornamento dell'inventario).
9. Il Sistema genera la **ricevuta**.
 - Il Cliente va via con la ricevuta e gli articoli acquistati.

Estensioni (o Flussi alternativi):

..

7a. Pagamento in contanti:

1. Il Cassiere inserisce l'importo in contanti presentato dal Cliente.
2. Il Sistema mostra il resto dovuto e apre il cassetto della cassa.
3. Il Cassiere deposita il contante presentato e restituisce il resto in contanti al Cliente.
4. Il Sistema registra il pagamento in contanti.

Classi concettuali il dominio POS

<i>Sale</i>	<i>(Vendita)</i>
<i>CashPayment</i>	<i>(Pagamento in contanti)</i>
<i>SalesLineItem</i>	<i>(Riga di vendita per l'articolo)</i>
<i>Item</i>	<i>(Articolo)</i>
<i>Register</i>	<i>(Registratore di cassa)</i>
<i>Cashier</i>	<i>(Cassiere)</i>
<i>Customer</i>	<i>(Cliente)</i>
<i>Store</i>	<i>(Negozio)</i>
<i>ProductDescription</i>	<i>(Descrizione prodotto)</i>
<i>ProductCatalog</i>	<i>(Catalogo prodotti)</i>

Modello di dominio iniziale POS



Modello di dominio iniziale: Monopoly

MonopolyGame

Player

Piece

Die

Board

Square

<i>MonopolyGame</i>	<i>(Partita a Monopoly)</i>
<i>Player</i>	<i>(Giocatore)</i>
<i>Piece</i>	<i>(Segnalino)</i>
<i>Die</i>	<i>(Dado)</i>
<i>Board</i>	<i>(Tabellone)</i>
<i>Square</i>	<i>(Casella)</i>

Oggetti rapporto

È comune avere dei dubbi (e commettere errori) nella scelta delle classi candidate

- ad es., ci vuole una classe Receipt (ricevuta)?
- gli errori saranno corretti nelle iterazioni successive

Utilizzare i termini del dominio

Crea un modello di dominio usando lo spirito con cui lavoro un cartografo

- Utilizza i nomi esistenti sul territorio. Es. se si sviluppa un modello per le prenotazioni aeree si assegna al cliente il nome Passeggero
- Escludere caratteristiche irrilevanti o fuori della portata. Es. il modello di dominio del Monopoly per l'iterazione 1 non vengono utilizzate le carte, quindi non è opportuno mostrare una classe “Card” nel modello per questa iterazione
- Non aggiungere cose che non ci sono

Modellare con classi descrizione

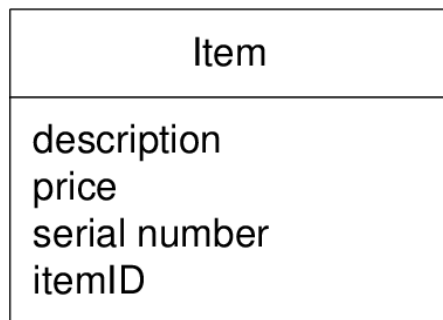
Una classe descrizione contiene informazioni che descrivono qualcos'altro

- *Item*: un oggetto fisico del negozio – potrebbe avere un numero di serie
- *ProductSpecification*: la specifica di un prodotto, ovvero la descrizione di un prodotto o di una tipologia di prodotti

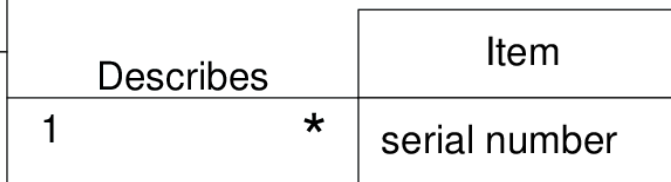
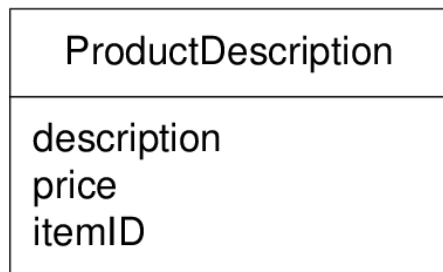
Può essere utile aggiungere classi descrizione per

- ridurre ridondanze e duplicazioni di informazioni
- evitare errori (a causa delle informazioni ripetute)

Modellare con classi descrizione



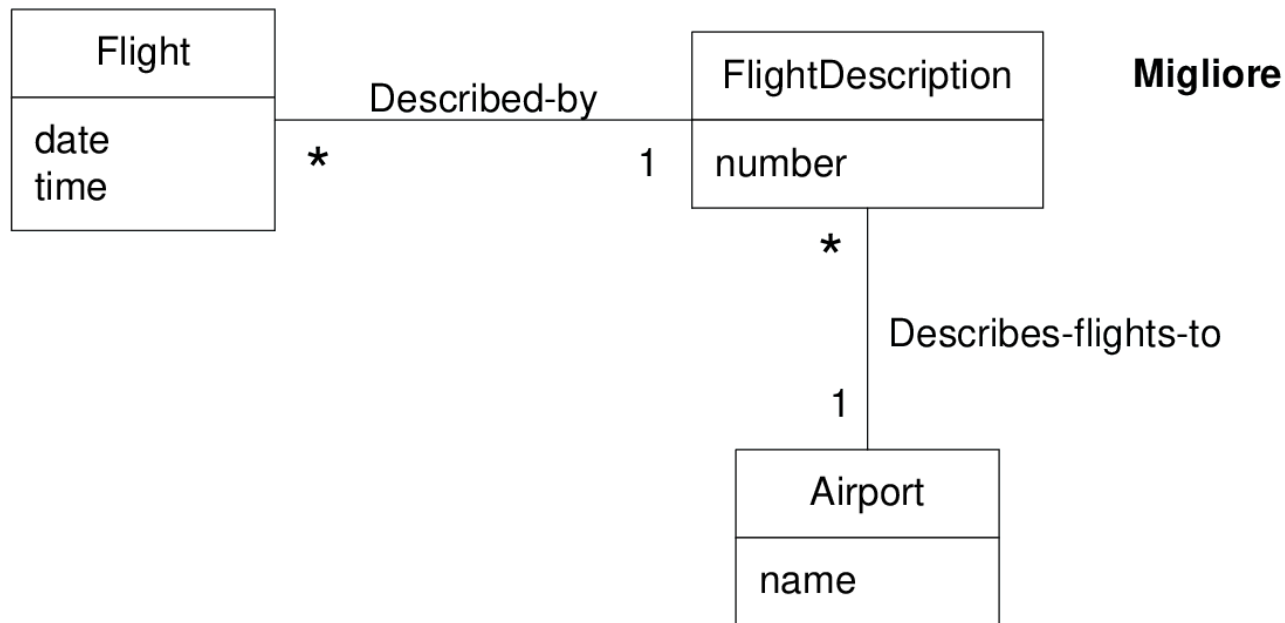
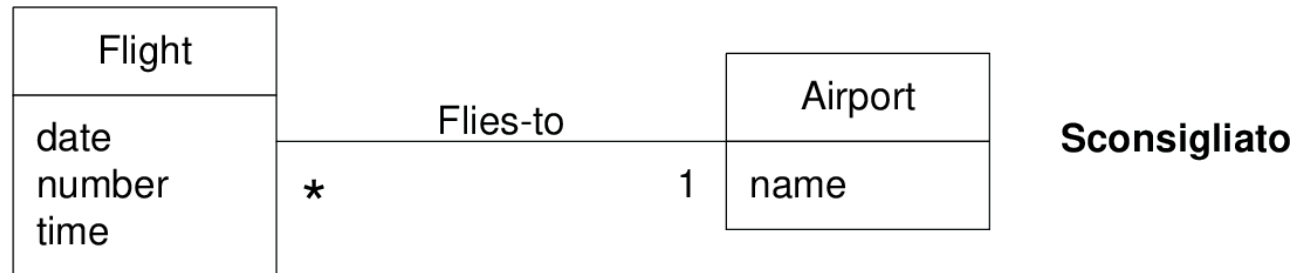
Sconsigliato



Migliore

La molteplicità indica che ciascun ProductDescription può descrivere molti item

Modellare con classi descrizione



Associazione

- Un'associazione è una relazione tra classi (più precisamente, tra istanze di queste classi) che indica una connessione significativa e interessante.
- in UML, una relazione semantica tra due o più classificatori che coinvolge connessioni tra le loro istanze

Quando mostrare un'associazione

- Quali associazioni mostrare?
 - *Solitamente quelle che implicano la conoscenza di una relazione che deve essere memorizzata per una certa durata di tempo (siano essi millisecondi o anni)*
- Per esempio
 - *dobbiamo ricordare quali istanze di SalesLineItem sono associate a un'istanza di Sale?*
 - *Dobbiamo ricordare su quale Square si trova un Piece?*
 - *dobbiamo ricordare il totale di quale Die un Player si è mosso in una Square?*
 - *dobbiamo ricordare che un Cashier ha acceduto una certa ProductDescription?*
 - *dobbiamo ricordare quali Square contiene il Board?*

Quando mostrare un'associazione

- Considerare l'inclusione delle seguenti associazioni in un modello di dominio
 - *associazioni per cui la conoscenza della relazione deve essere conservata per qualche durata (associazioni da ricordare)*
 - *associazioni identificate mediante l'elenco di associazioni comuni*
- Evitare di inserire troppe associazioni in un modello di dominio

Elenco di associazioni comuni

Categoria	Esempi
A è una transazione correlata a un'altra transazione B	<i>CashPayment—Sale Cancellation—Reservation</i>
A è un elemento/riga di una transazione B	<i>SalesLineItem—Sale</i>
A è un prodotto o servizio per una transazione (o riga per l'articolo) B	<i>Item—SalesLineItem (o Sale) Flight—Reservation</i>
A è un ruolo relativo a una transazione B	<i>Customer—Payment Passenger—Ticket</i>
A è una parte fisica o logica di B	<i>Drawer—Register Square—Board Seat—Airplane</i>
A è contenuto fisicamente o logicamente in B	<i>Register—Store, Item—Shelf Square—Board Passenger—Airplane</i>
A è una descrizione per B	<i>ProductDescription—Item FlightDescription—Flight</i>

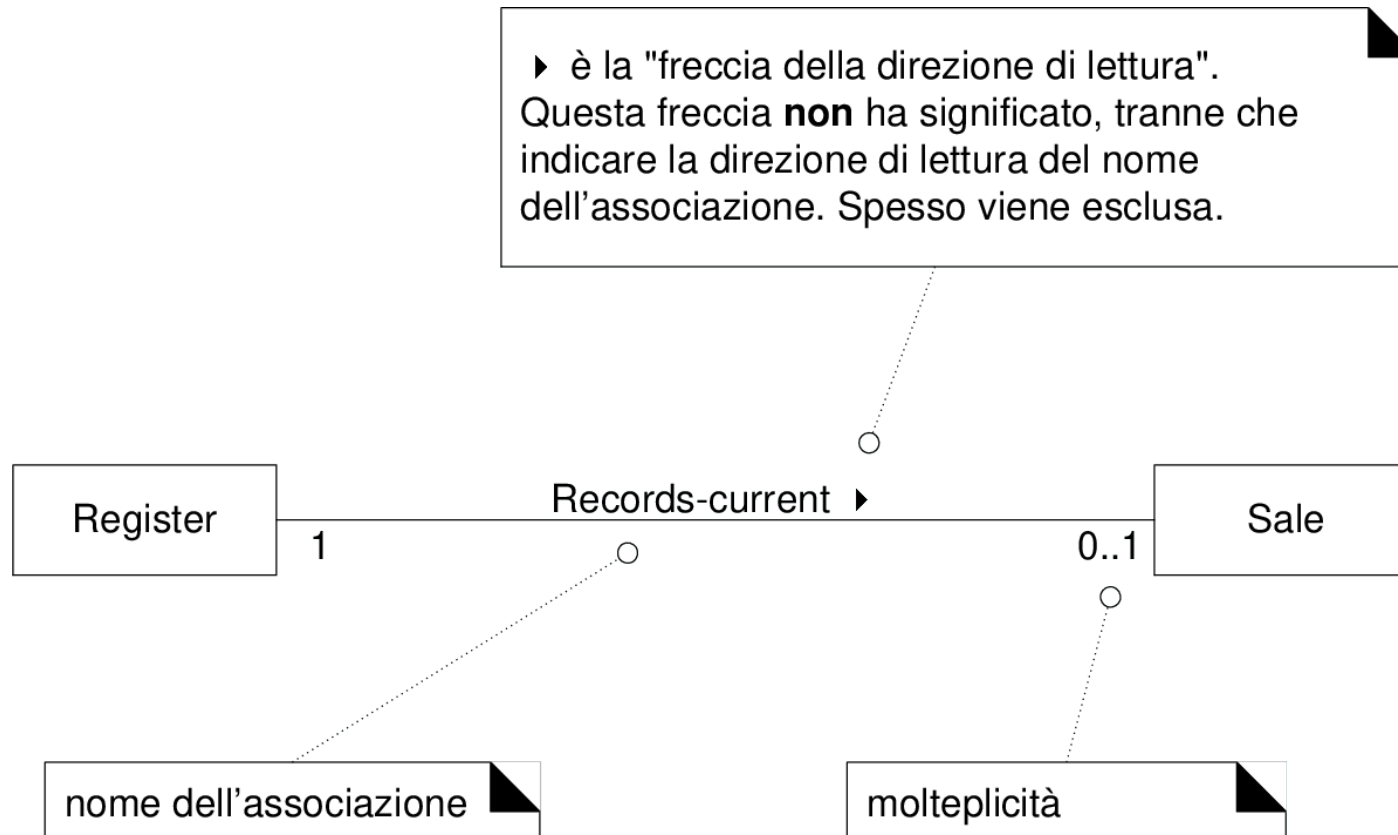
Elenco di associazioni comuni

A è un membro di B	<i>Cashier—Store Player—MonopolyGame Pilot—Airline</i>
A è una sottounità organizzativa di B	<i>Department—Store Maintenance—Airline</i>
A utilizza o gestisce o possiede B	<i>Cashier—Register Player—Piece Pilot— Airplane</i>
A è vicino/prossimo a B	<i>SalesLineItem—SalesLineItem Square —Square City—City</i>

Le Associazioni saranno implementate nel software?

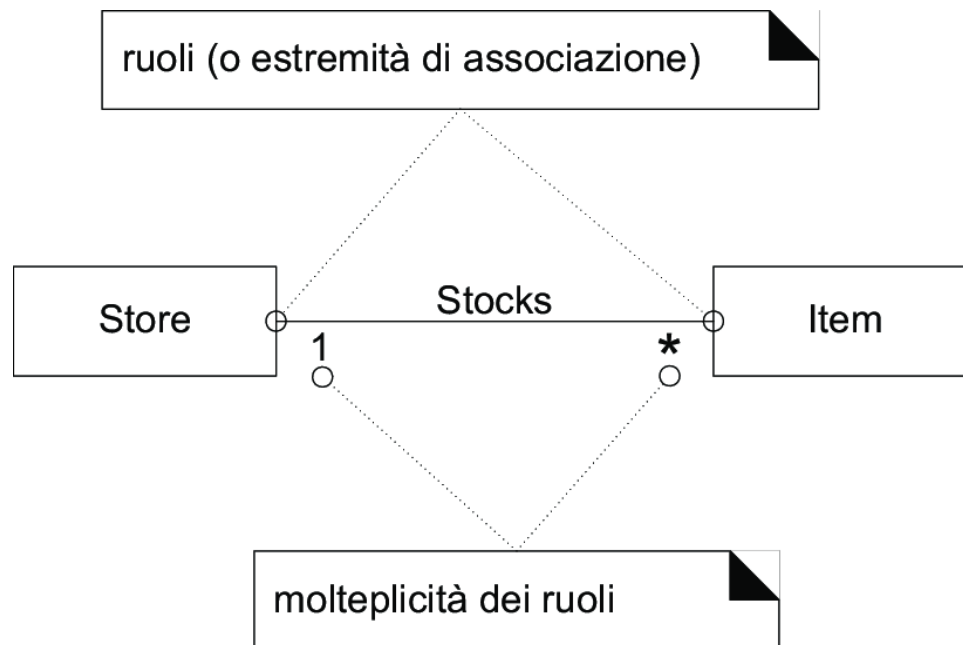
- Le associazioni del modello di progetto andranno implementate
- Un modello di dominio non è un modello di progetto
 - *una associazione descrive una relazione significativa tra oggetti del mondo reale*
 - *non descrive caratteristiche di oggetti software*

Notazione UML



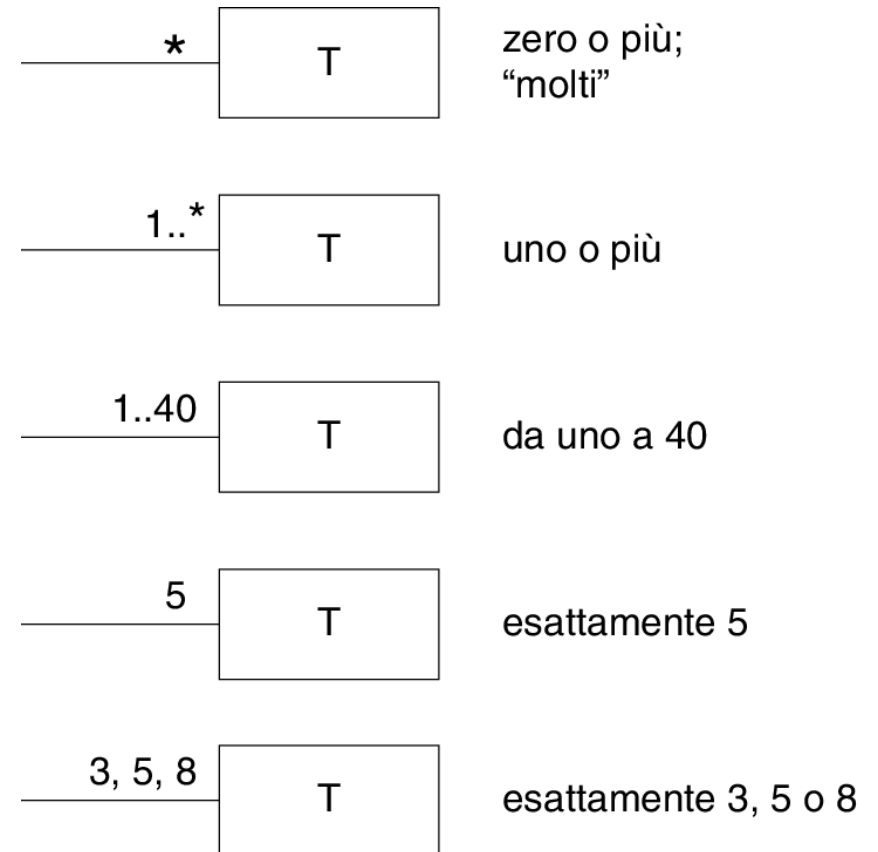
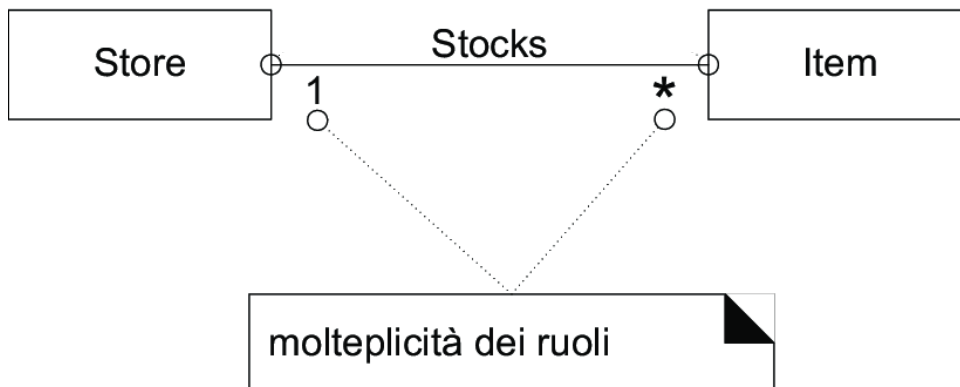
Ruoli

- Ciascuna estremità di una associazione è chiamata un **ruolo** (estremità di associazione)
- I ruoli possono avere
 - Molteplicità
 - Nome
 - Navigabilità



Molteplicità

- La molteplicità di un ruolo indica quante istanze di una classe possono essere associate a una istanza dell'altra classe



Nome di ruolo



La molteplicità dipende dal contesto



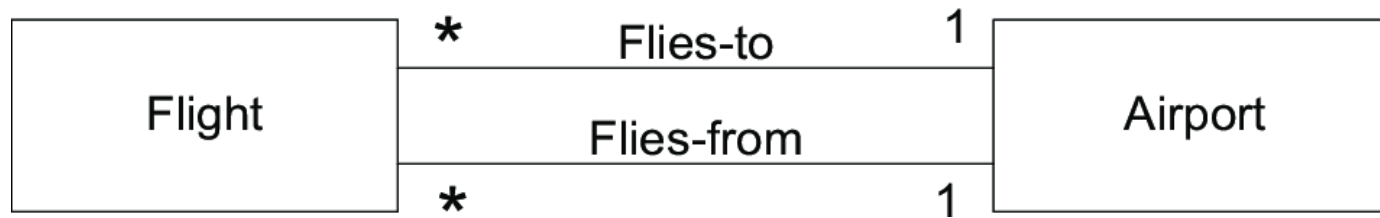
La molteplicità deve essere "1" oppure "0..1"?

La risposta dipende dall'interesse che si ha nell'utilizzo del modello. Normalmente e praticamente, la molteplicità indica un vincolo del dominio che ci interessa poter controllare nel software, se questa relazione viene implementata o è riflessa in oggetti software o in una base di dati. Per esempio, un particolare articolo potrebbe essere venduto o scartato, quindi non essere più disponibile in magazzino. Da questo punto di vista, "0..1" è logico, ma...

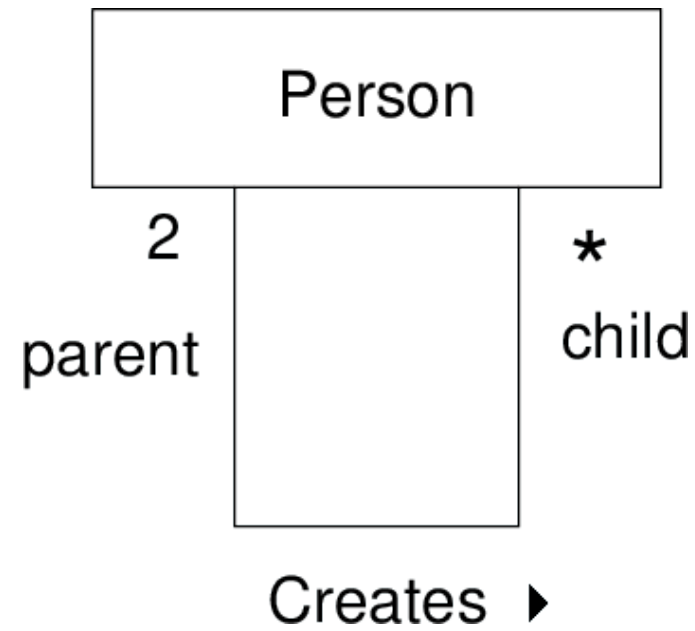
Ci interessa quel punto di vista? Se questa relazione fosse implementata nel software, probabilmente vorremmo assicurarci che ciascuna istanza software di *Item* sia sempre correlata a una particolare istanza di *Store*; se ciò non è vero, vuol dire che si è verificato un guasto o un errore negli elementi software o nei dati.

Questo modello di dominio parziale non rappresenta oggetti software, ma le molteplicità registrano dei vincoli il cui valore pratico è normalmente correlato al nostro interesse per la costruzione del software o delle basi di dati (che riflettono il dominio del nostro mondo reale) e dei relativi controlli di validità. Da questo punto di vista, "1" può essere il valore desiderato.

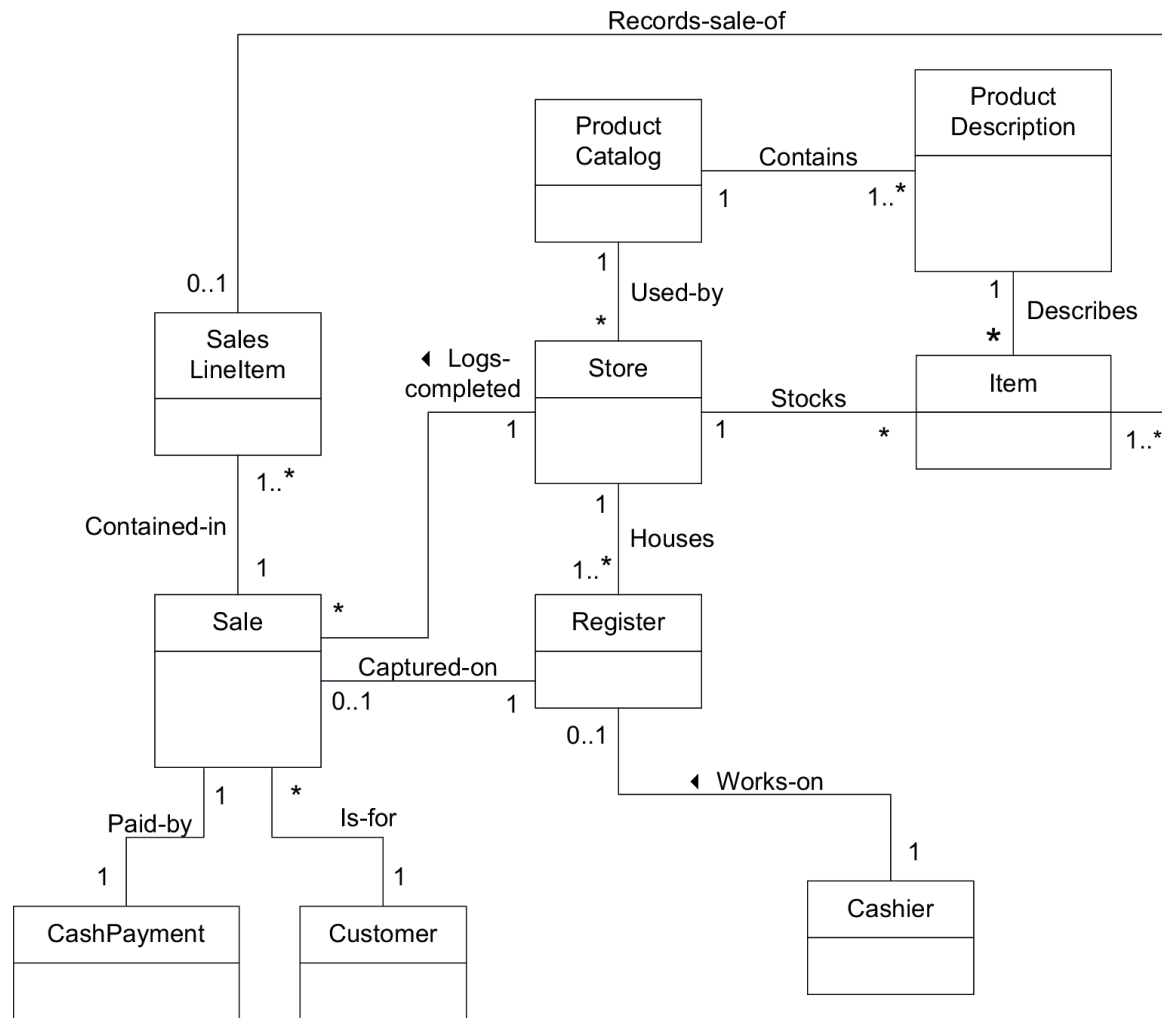
Associazioni multiple tra classi



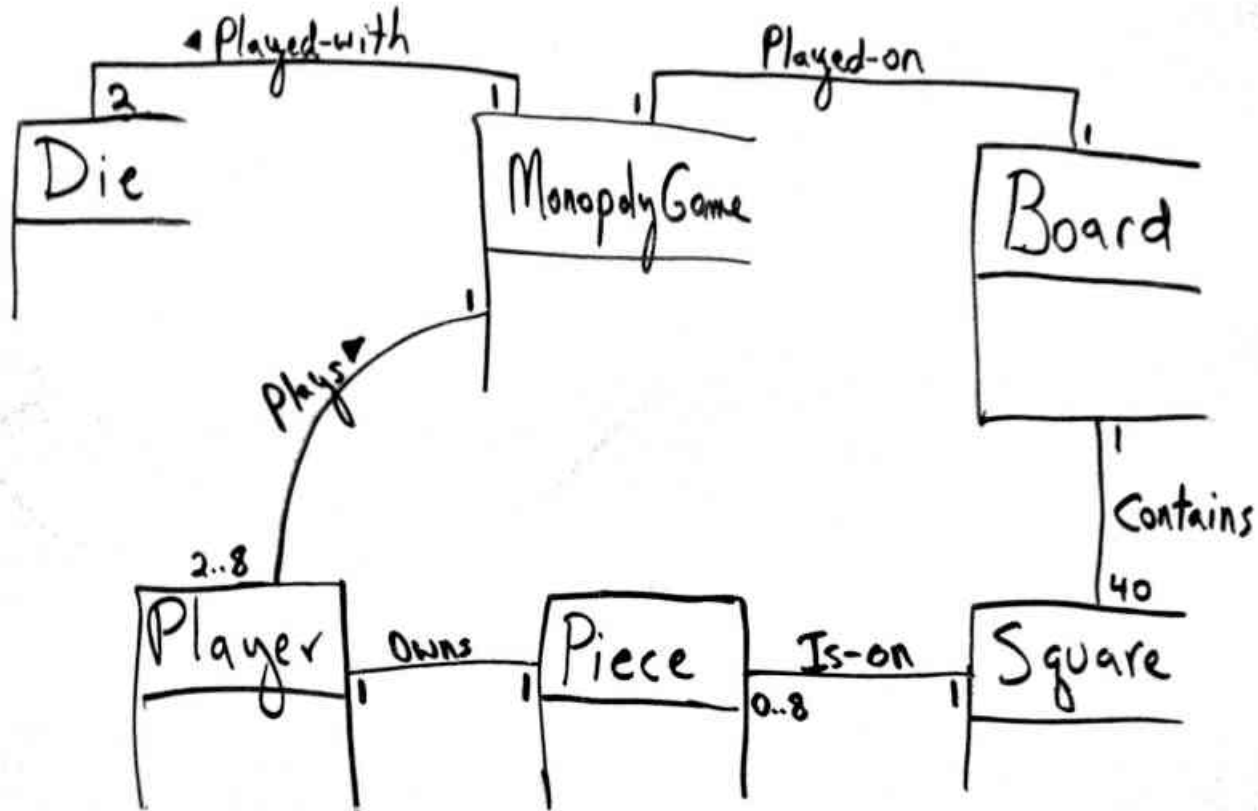
Associazioni riflesse



Modello di dominio parziale per POS NextGen



Modello di dominio parziale per Monopoly



Aggregazione e composizione

Aggregazione

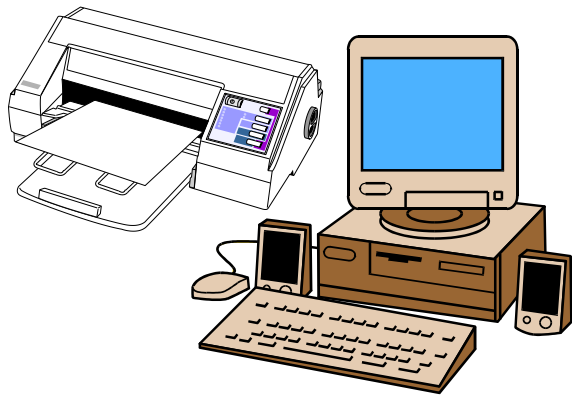
- una associazione che rappresenta una relazione intero -parte

Composizione (aggregazione composta)

- una forma forte di aggregazione in cui
 - *una parte appartiene a un composto alla volta*
 - *ciascuna parte appartiene sempre a un composto*
 - *il composto è responsabile della creazione e cancellazione delle sue parti*
- ad es., Board-Square

Aggregazione e composizione

Aggregazione



Alcuni oggetti sono debolmente collegati, come un computer e le sue periferiche

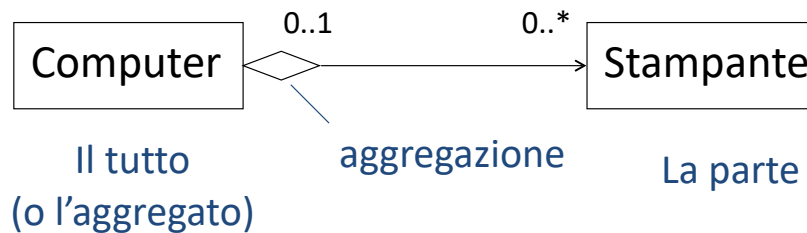
Composizione



Alcuni oggetti sono fortemente collegati, come un albero e le sue foglie

Aggregazione

L'Aggregazione è una relazione del tipo *tutto-parte*



Un Computer può essere collegato a 0 o più Stampanti

In qualunque istante una Stampante è collegata a 0 o 1 Computer

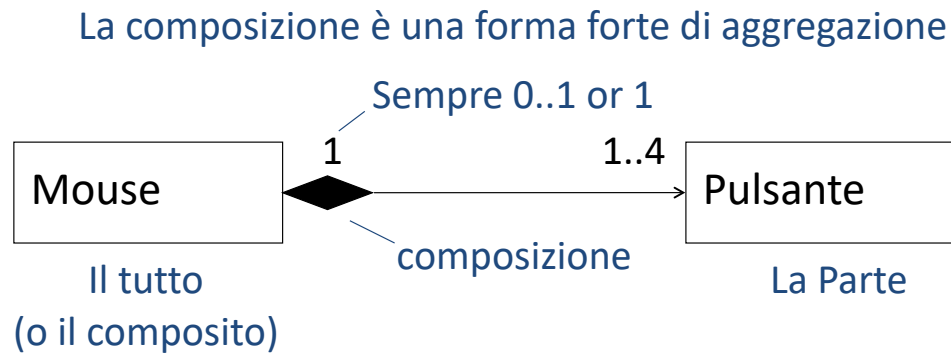
Nel corso del tempo, molti Computer possono utilizzare la stessa Stampante

La Stampante può esistere anche senza essere collegata ad alcun Computer

La Stampante è indipendente dal Computer

- L'aggregato può in alcuni casi esistere indipendentemente dalle parti, ma in altri casi no
- Le parti possono esistere indipendentemente dall'aggregato
- L'aggregato è in qualche modo incompleto se mancano alcune delle sue parti
- È possibile che più aggregati condividano una stessa parte

Composizione



Gli oggetti Pulsante non hanno una propria esistenza. Distruggendo l'oggetto Mouse, vengono distrutti anche gli oggetti Pulsante, che ne costituiscono parte integrante.

Ogni oggetto Pulsante può appartenere esclusivamente ad un solo oggetto Mouse

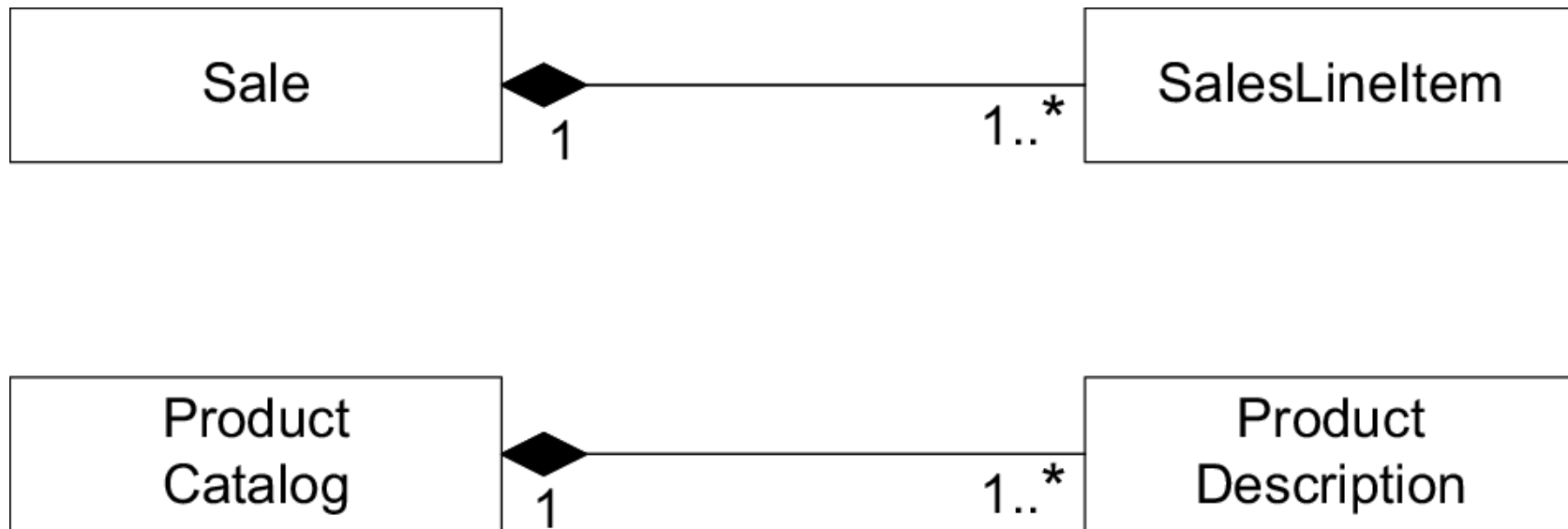
- Ogni parte può appartenere a un solo composito per volta
- Il composito è l'unico responsabile di tutte le sue parti: questo vuol dire che è responsabile della loro creazione e distruzione
- Il composito può rilasciare una sua parte, a patto che un altro oggetto si prenda la relativa responsabilità
- Se il composito viene distrutto, deve distruggere tutte le sue parti o cederne la responsabilità a qualche altro oggetto
- La composizione è transitiva e asimmetrica

Come identificare la composizione

Si consideri di mostrare una composizione quando:

- c'è un ovvio gruppo fisico o logico intero-parte
- la vita della parte è limitata dalla vita del composto
 - *c'è una dipendenza di creazione/cancellazione*
- alcune proprietà dell'intero si propagano anche alle parti (ad es., posizione)
- alcune operazioni dell'intero si propagano anche alle parti (ad es., creazione/cancellazione, movimento)

Composizioni nell'applicazione POS



Attributi

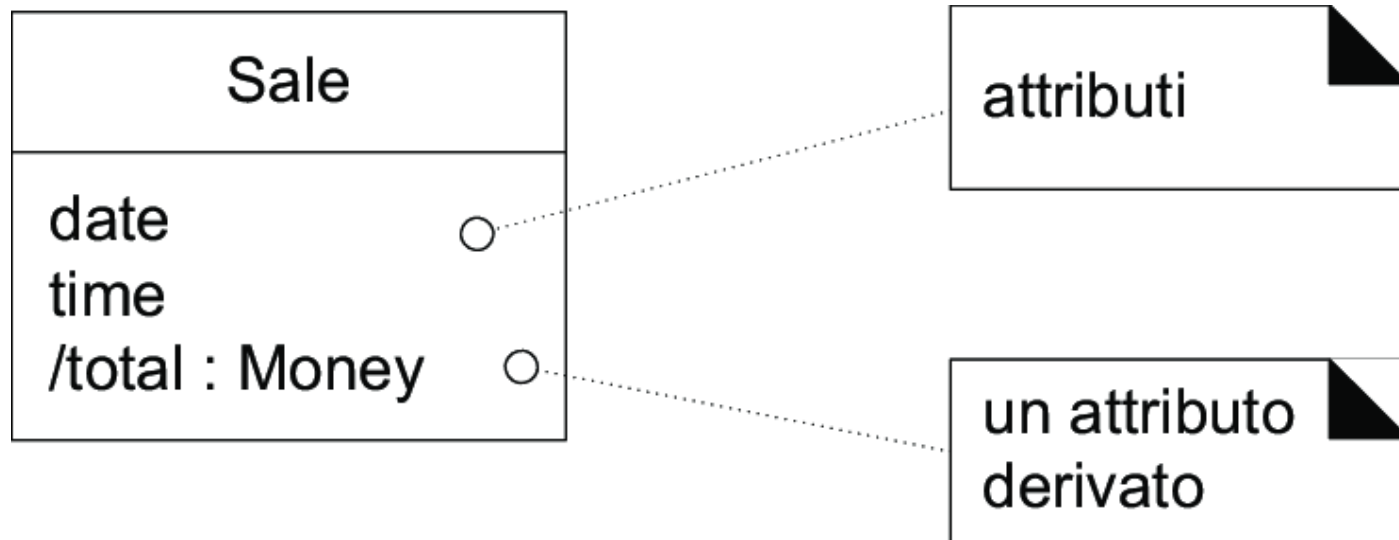
Un attributo

- è una proprietà elementare degli oggetti di una classe
- a cui ciascun oggetto della classe associa un valore

Un modello di dominio deve contenere gli attributi per cui i requisiti indicano la necessità di ricordare informazioni

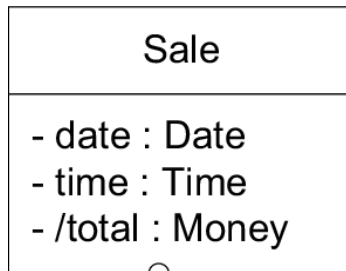
- ad es., una ricevuta (che contiene le informazioni di una vendita) deve contenere, tra l'altro
 - *data e ora della vendita*
 - *nome e indirizzo del negozio*
 - *ID del cassiere*
- pertanto
 - la classe *Sale* deve avere un attributo *dateTime*
 - la classe *Store* deve avere attributi *name* e *address*
 - la classe *Cashier* deve avere un attributo *ID*

Classe e attributi

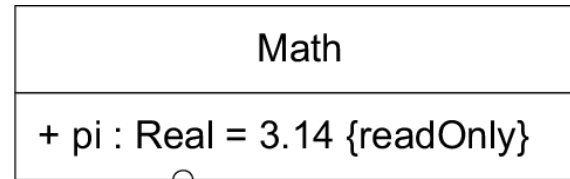


Notazione UML per gli attributi

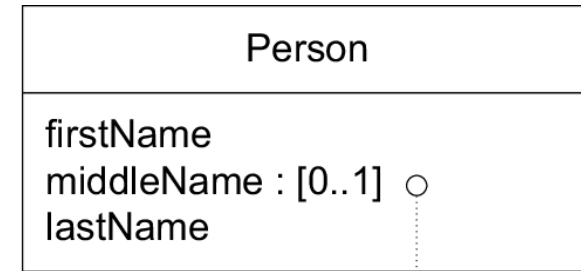
visibility name : type multiplicity = default { property-string }



attributi con
visibilità privata



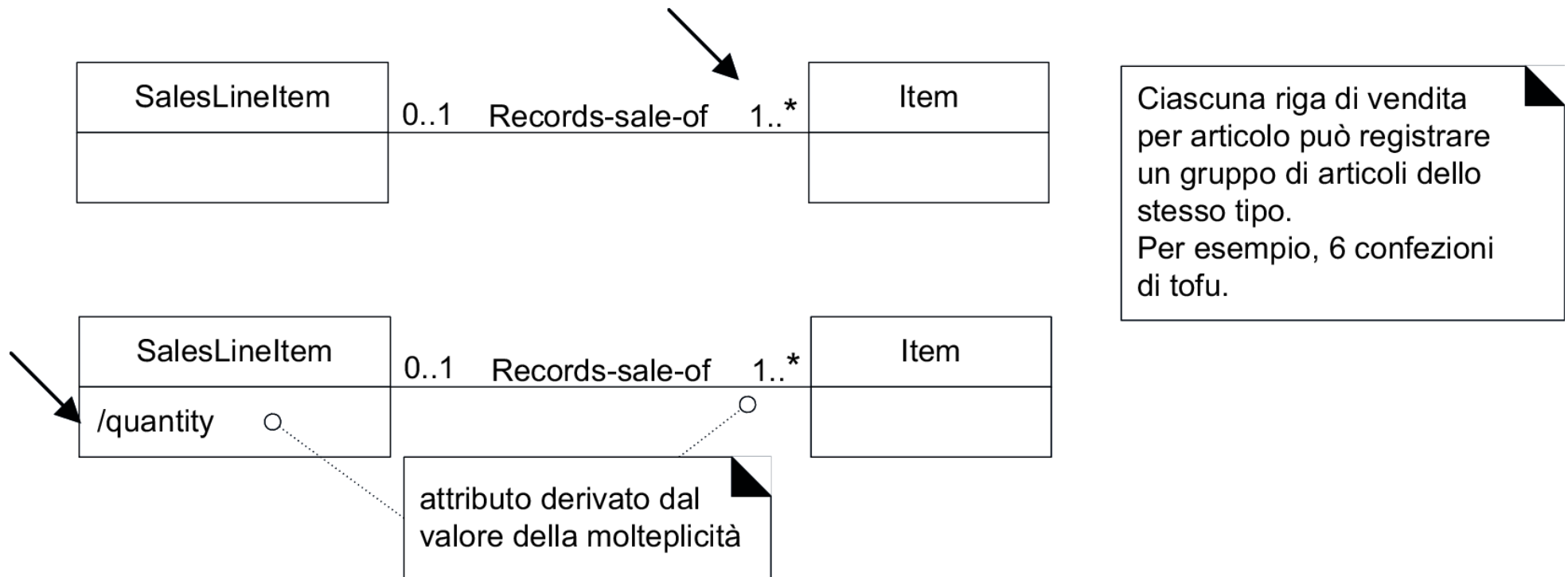
attributo con visibilità pubblica, di
sola lettura e con inizializzazione



valore opzionale

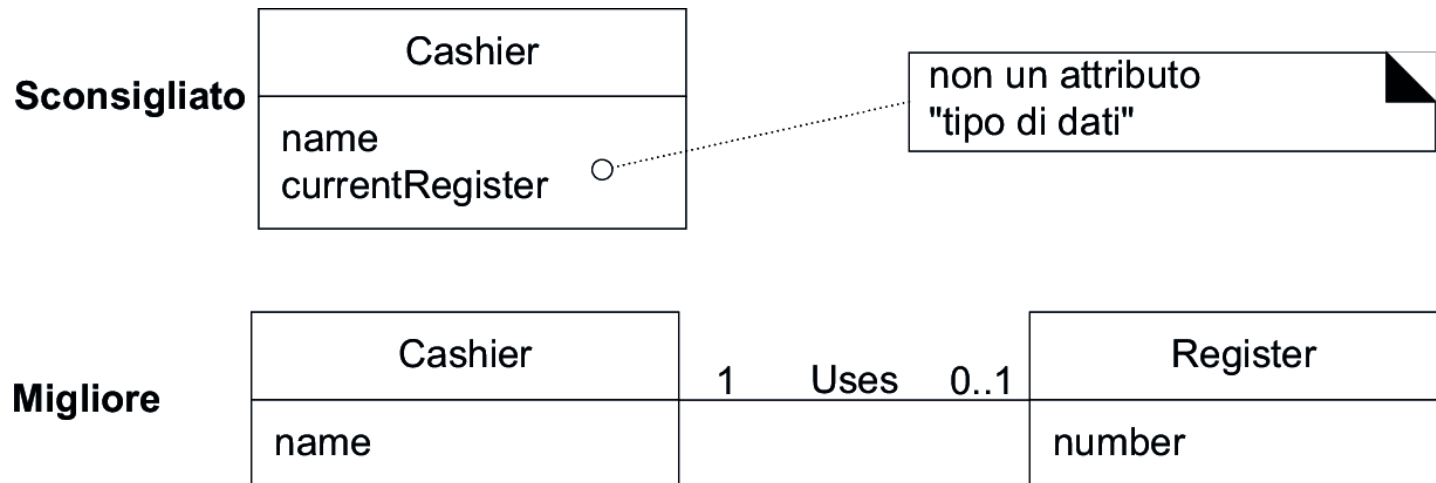
Attributi Derivati

- Può essere utile indicare attributi (e associazioni) che possono essere calcolati (derivati) da altre informazioni



Tipi di attributi appropriati nel modello di dominio

- Gli attributi devono avere tipo semplice, elementare, corrispondente a un tipo di dati primitivo
- Gli attributi non devono avere tipo corrispondente a un concetto complesso del dominio



Attributo oppure classe concettuale e associazioni

Sale
store

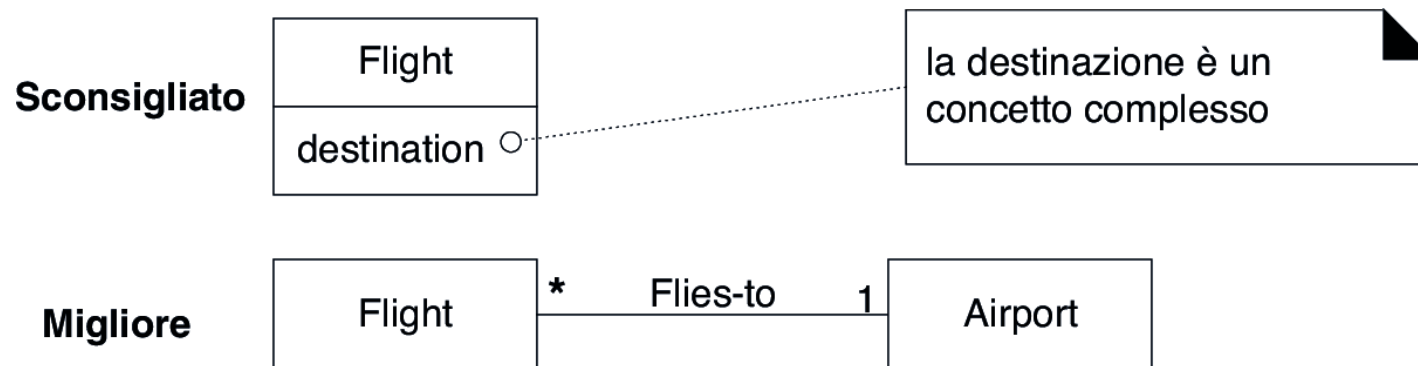
oppure... ?

Sale

Store
phoneNumber

Attributo oppure classe concettuale e associazioni

- Classi concettuali vanno correlate con un'associazione, non con un attributo



Tipi di dato

Il tipo degli attributi deve essere un tipo di dato

- in UML, un **tipo di dato** è un insieme di valori in cui l'identità univoca non è significativa
 - ad es, 5, 'abc' o un numero di telefono sono valori
 - le *Persone* non sono un tipo di dato
- altre caratteristiche dei tipi di dato
 - uguaglianza diversa da identità
 - valori immutabili (l'istanza "5" di Integer è immutabile, un'istanza di *Person* potrebbe cambiare il proprio *lastName*)

Il tipo degli attributi deve essere un tipo di dato

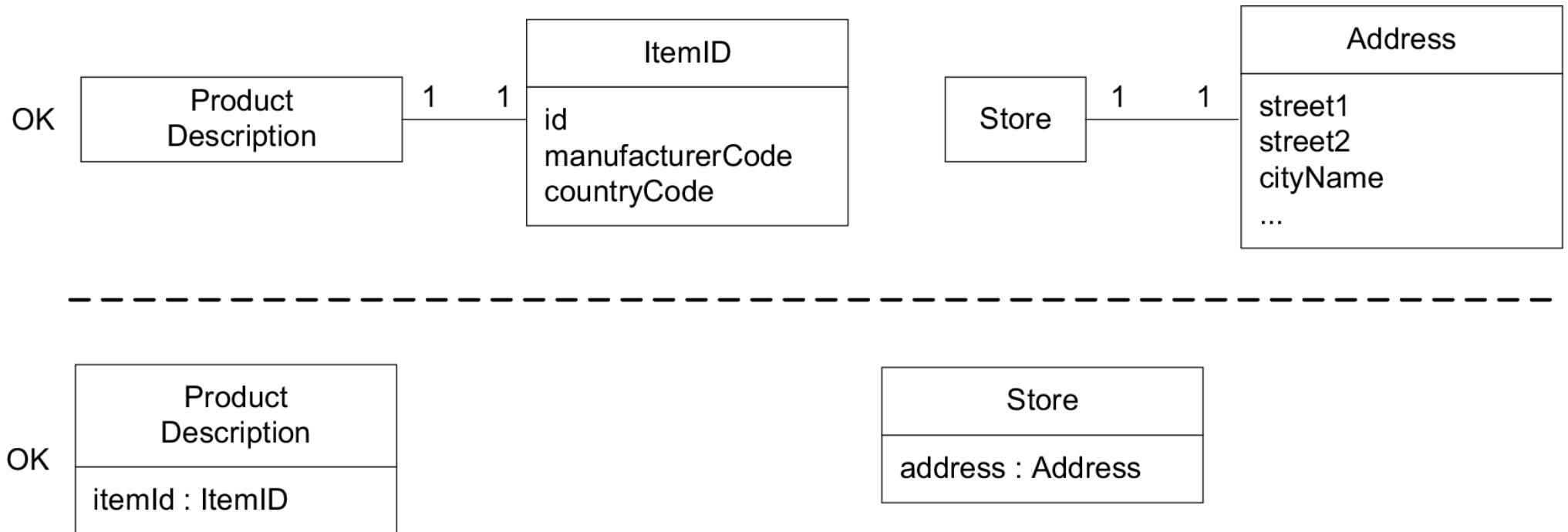
- in caso di dubbio, definisci una nuova classe concettuale e una associazione

Classi tipo di dati

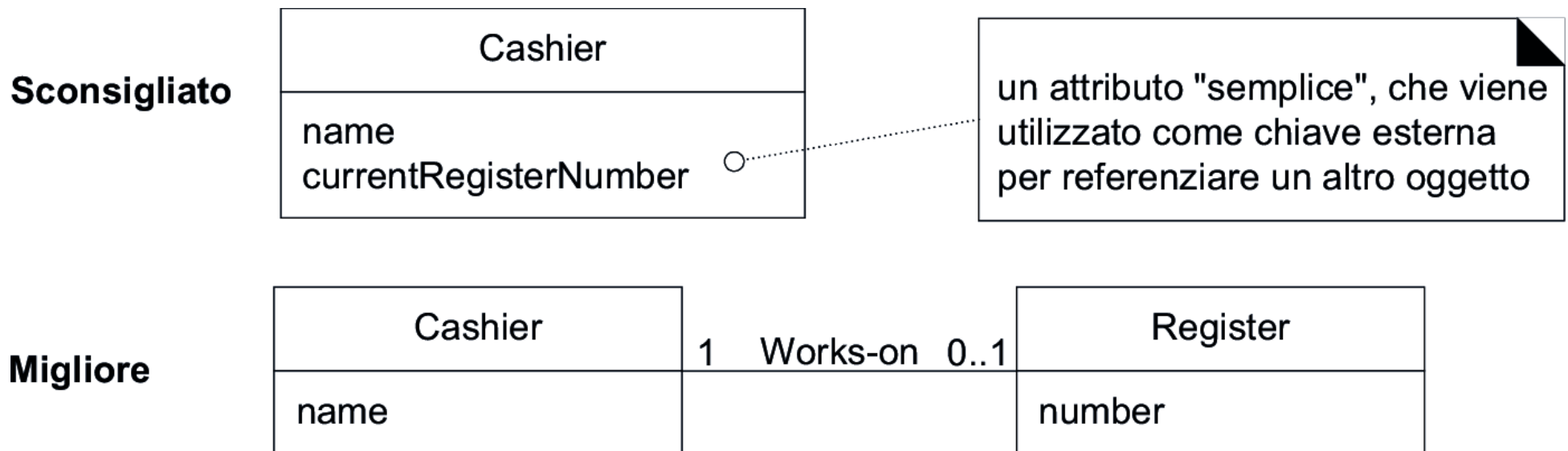
Presentare nel modello di dominio con una nuova classe tipo di dato ciò che inizialmente può essere considerato un numero una stringa se:

- valore composto da più sezioni
 - es. numero di telefono, nome di persona
- ci sono operazioni associate al valore, come il parsing o la convalida
 - Numero di previdenza sociale, codice fiscale
- presenza di ulteriori attributi
 - Un prezzo promozionale potrebbe avere una data di inizio e una data di fine
- quantità con unità di misura
 - L'importo di un pagamento ha un'unità monetaria
- astrazione di uno o più tipi
 - Il codice identificativo di un articolo nel dominio delle vendite è una generalizzazione di tipi come l'Universal Product Code (UPC) e European Article Number (EAN)

Due modi per indicare proprietà di un tipo di dato in un oggetto



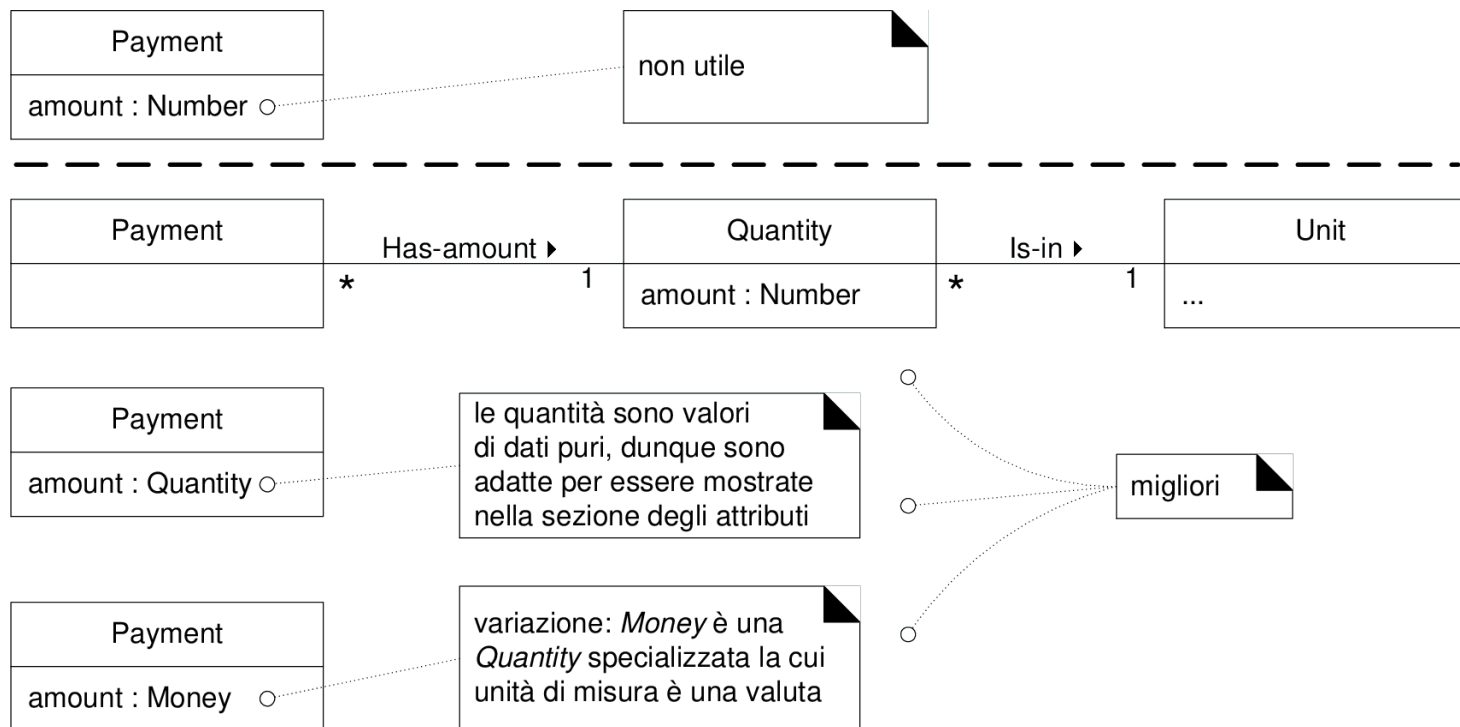
Non è opportuno utilizzare gli attributi come chiavi esterne



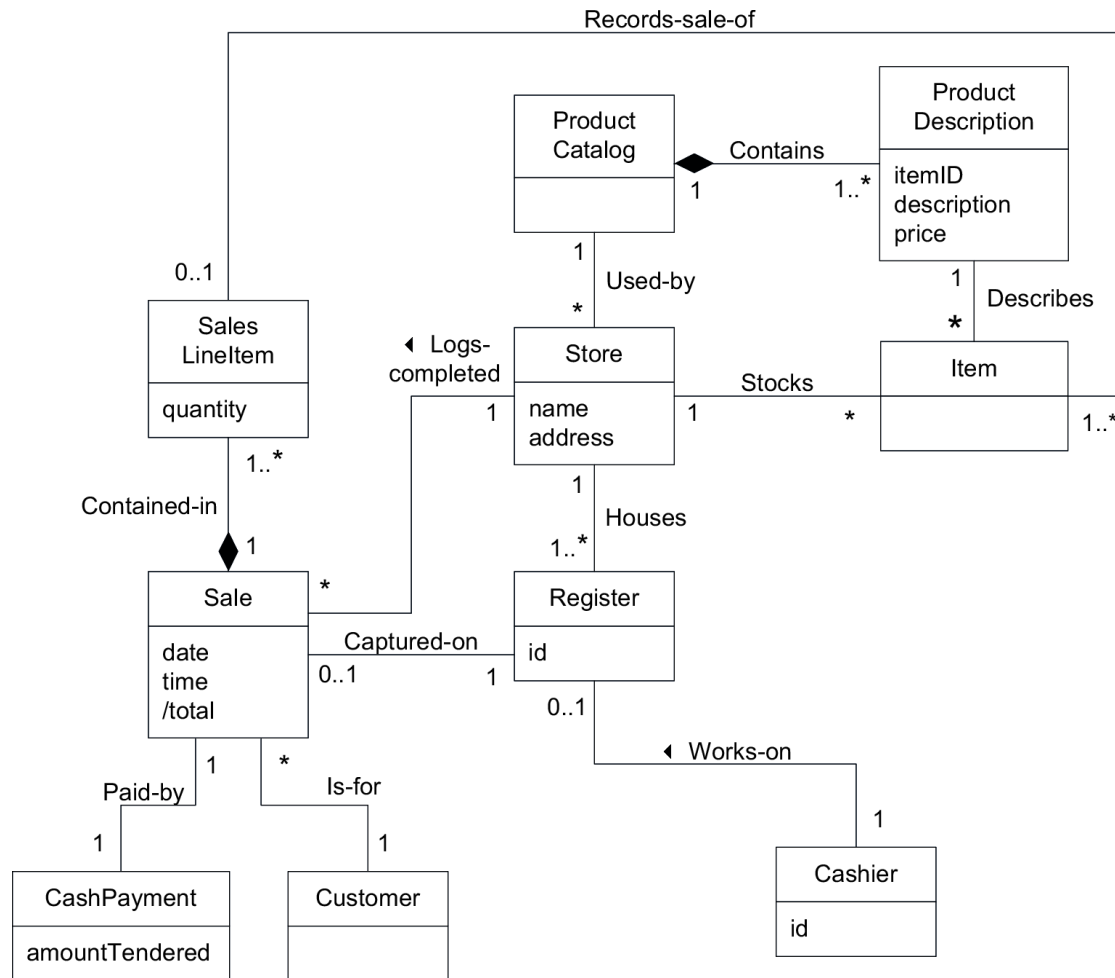
Modellare quantità e unità di misura

La maggior parte delle quantità numeriche non dovrebbero essere rappresentate come dei semplici numeri.

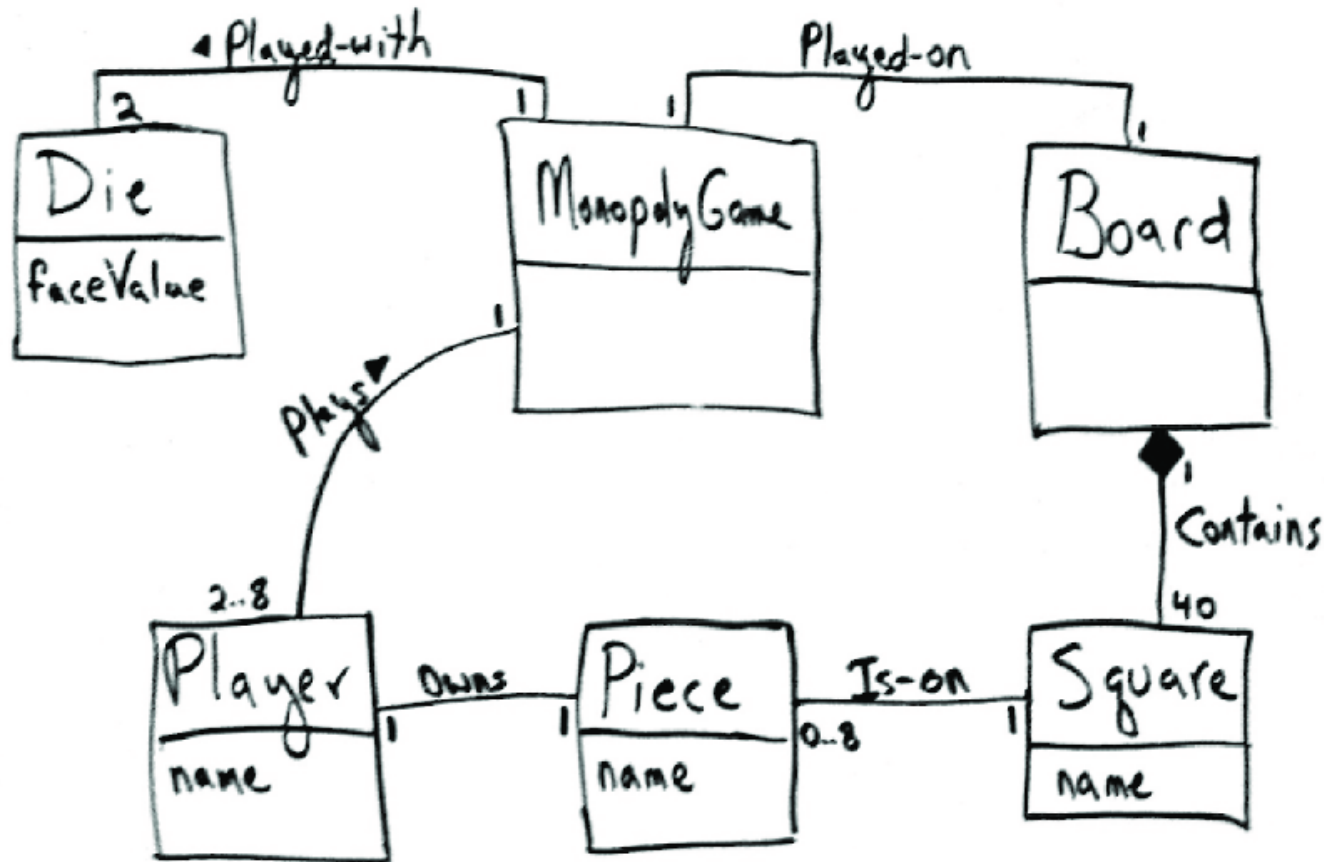
- Si consideri un prezzo un peso: dire il prezzo è 13 o il peso è 37 non significa granché.



Modello di dominio parziale per POS NextGen



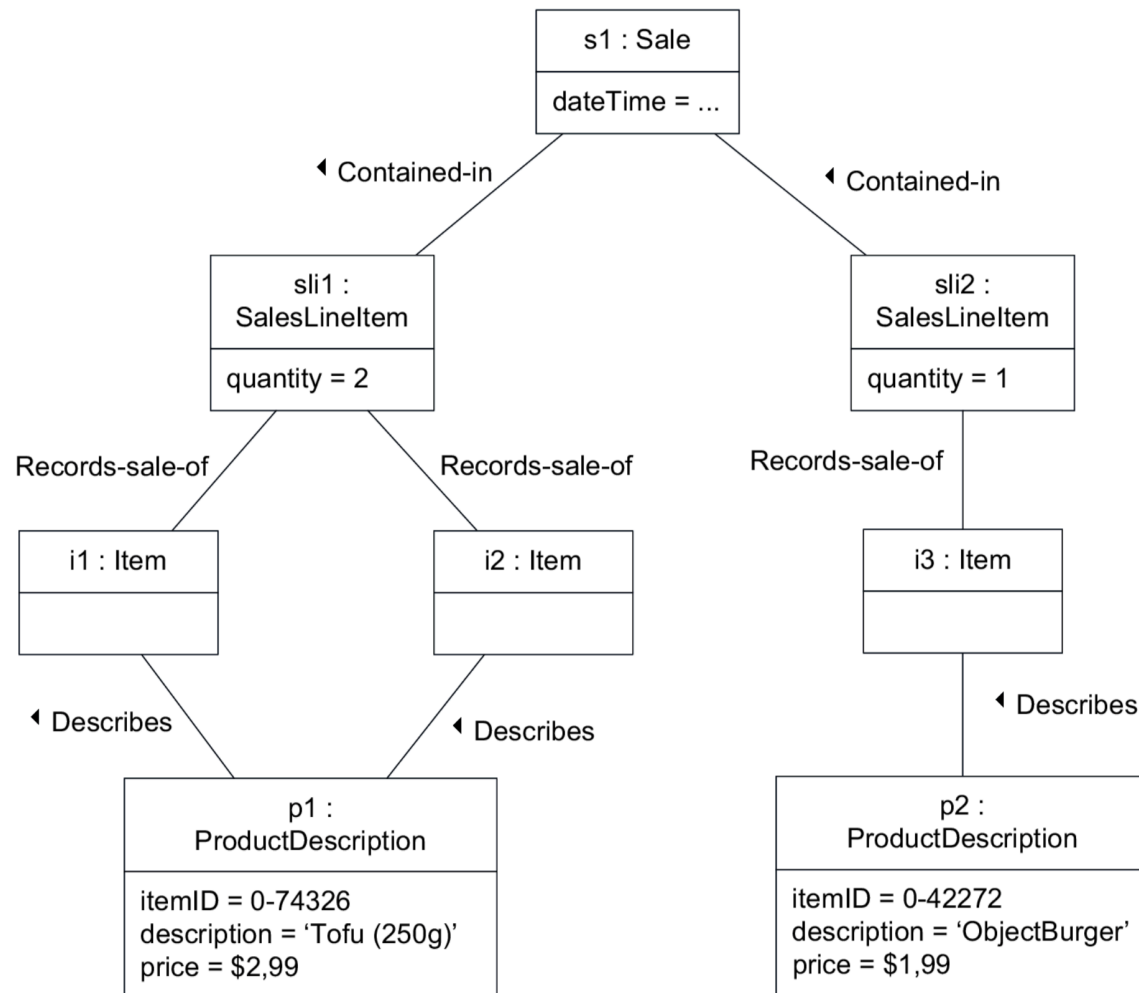
Modello di dominio parziale per Monopoly



Il modello di dominio è corretto?

- Non esiste un solo modello di dominio corretto
- Tutti i modelli sono approssimazioni del dominio che si sta tentando di capire
- Una domanda migliore è: il modello di dominio è utile?

Diagramma degli oggetti di domino



Modellazione di dominio iterativa ed evolutiva

- Si eviti un grosso sforzo in valutazione secondo l'approccio a cascata per creare un modello di dominio completo e corretto
- Si limiti la modellazione di dominio a non più di alcune ore per ogni iterazione

Modelli di dominio in UP

Disciplina	Elaborato Iterazione →	Ideaz. I1	Elab. E1..En	Costr. C1..Cn	Trans. T1..T2
Modellazione del business	Modello di Dominio		i		
Requisiti	Modello dei Casi d'Uso (SSD)	i	r		
	Visione	i	r		
	Specifiche Supplementari	i	r		
	Glossario	i	r		
Progettazione	Modello di progetto		i	r	
	Documento dell'Architettura Software		i	r	
	Modello dei Dati		i		